Universidad de San Carlos de Guatemala



Centro Universitario de Occidente

División de Ciencias de la Ingeniería

Área Profesional

Lab. Redes de computadoras 2

Ing. Juan Francisco Rojas

Aux. Yefri González

“Proyecto 2 - XelajuNetwork Informe Individual Estudiante 3 Multiwan”

Brígido Josías Alvarado Pec | 202230251

Quetzaltenango, 13 de octubre del 2026

“Id y enseñad a todos”

Índice

Índice **2**
Configuración en R-EDGE Multi-WAN **3**
1. Descripción del componente 3
2. Creación y administración de la máquina virtual 3
3. Asignación de dispositivos físicos a la VM 6
3.1 Teléfono Samsung — ISP1 6
3.2 Teléfono LG — ISP2 6
3.3 Adaptador Ethernet hacia el firewall 7
4. Nombres persistentes de las WAN 9
5. Comprobación y asignación de direcciones WAN 10
6. Configuración persistente de red 11
6.1 Archivo principal 11
6.2 Interfaces Multi-WAN 11
6.3 Enlace de tránsito hacia el firewall 13
7. Reenvío IPv4 16
8. Parámetros generales del Multi-WAN 17
9. Archivo externo de políticas 18
10. NAT y reenvío de WireGuard 20
11. Aplicación de políticas y balanceo 24
12. Monitor y failover automático 27
13. Servicio persistente de Multi-WAN 30
14. Salida del agente de playit por el Multi-WAN 31
15. Agente de Zabbix 34
16. Comandos de verificación 35
16.1 Interfaces y rutas 35
16.2 Salida individual por cada proveedor 36
16.3 Políticas y NAT 36
16.4 Estado y registros 36
Archivos de Configuración **37**
ÁRBOL DE ARCHIVOS DE CONFIGURACIÓN DE R-EDGE 37
DESCRIPCIÓN DE LOS ARCHIVOS 38
Pruebas Individuales **41**

Informe Estudiante 3

Configuración en R-EDGE Multi-WAN

1. Descripción del componente

El nodo R-EDGE proporciona la salida a Internet de XelajuNetwork mediante dos proveedores independientes. Sus funciones son:

- Mantener dos enlaces WAN activos.
- Aplicar políticas de salida según dirección de origen, puerto y protocolo.
- Distribuir entre ambos ISP las conexiones que no tienen una política específica.
- Realizar NAT de salida.
- Detectar automáticamente la caída y recuperación de cada ISP.
- Registrar la política aplicada y la interfaz utilizada realmente.
- Entregar a Zabbix las métricas de entrada y salida de ambos enlaces.
- Integrar el punto de entrada público utilizado por la VPN.

La implementación se realizó en una máquina virtual Debian administrada con libvirt/QEMU. Los dos proveedores son teléfonos conectados por USB:

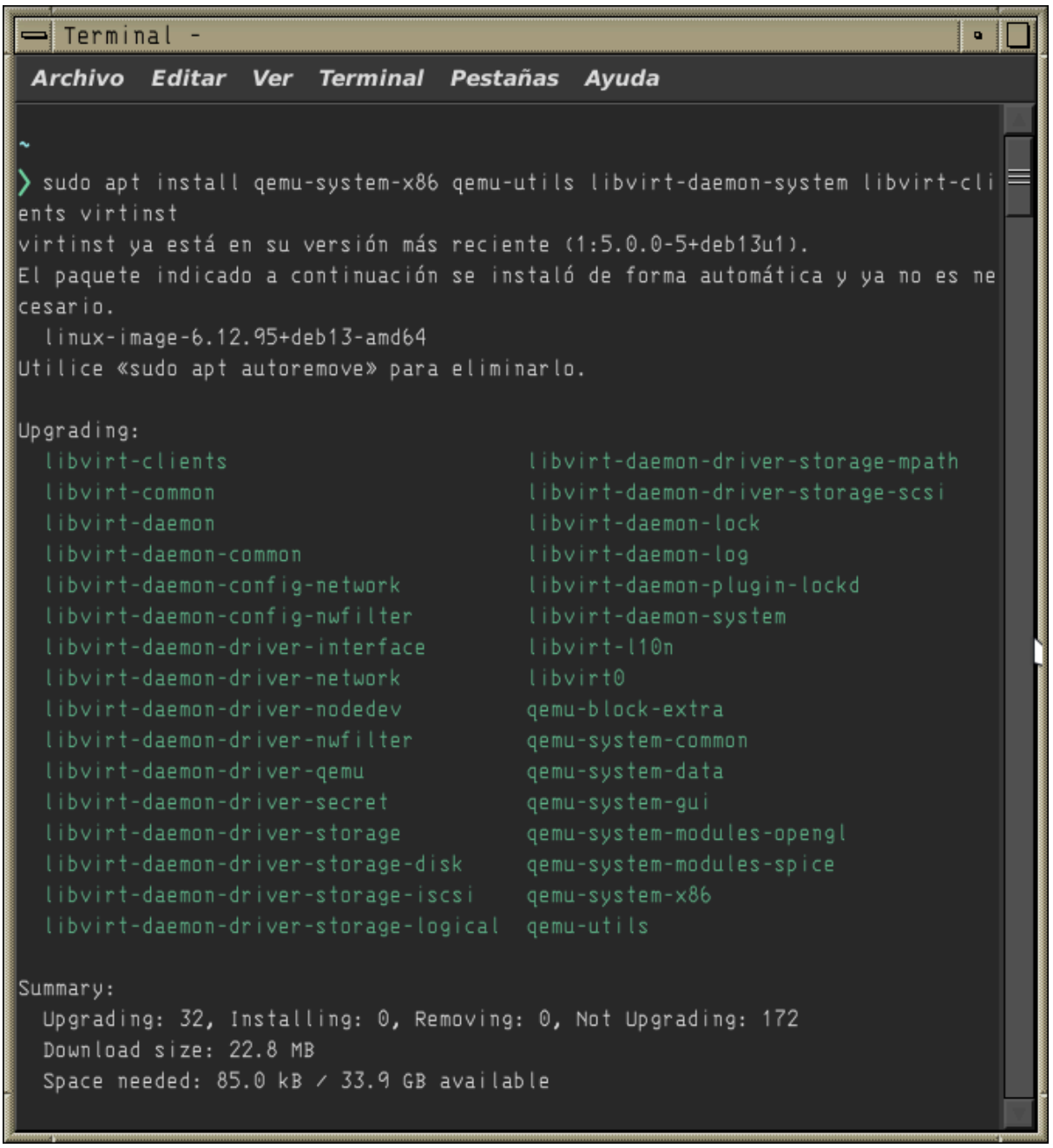
2. Creación y administración de la máquina virtual

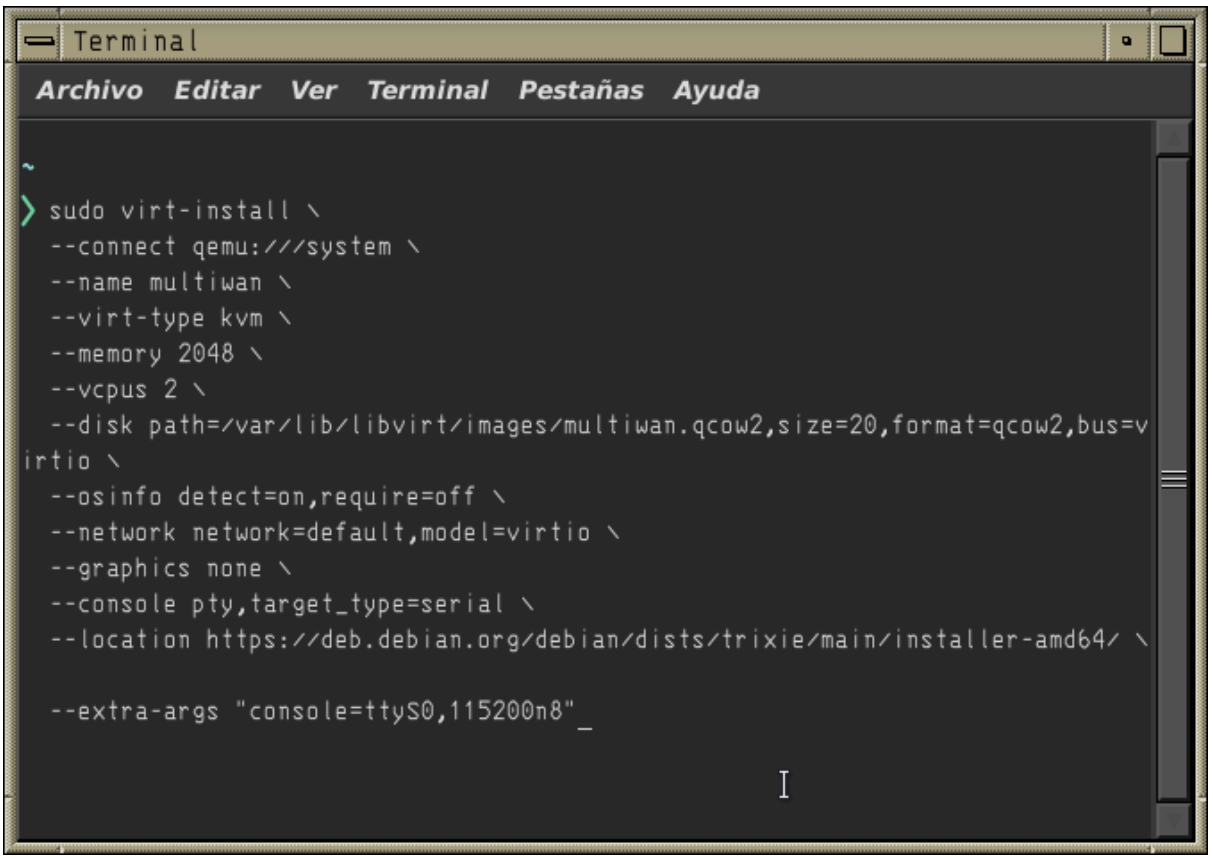
La VM se creó con Debian en modo texto:

```
sudo virt-install \
  --connect qemu:///system \
  --name multiwan \
  --virt-type kvm \
  --memory 2048 \
  --vcpus 2 \
  --disk path=/var/lib/libvirt/images/multiwan.qcow2,size=5,format=qcow2,bus=virtio \
  --osinfo detect=on,require=off \
  --network network=default,model=virtio \
  --graphics none \
  --console pty,target_type=serial \
  --location https://deb.debian.org/debian/dists/trixie/main/installer-amd64/ \
  --extra-args "console=ttyS0,115200n8"
```

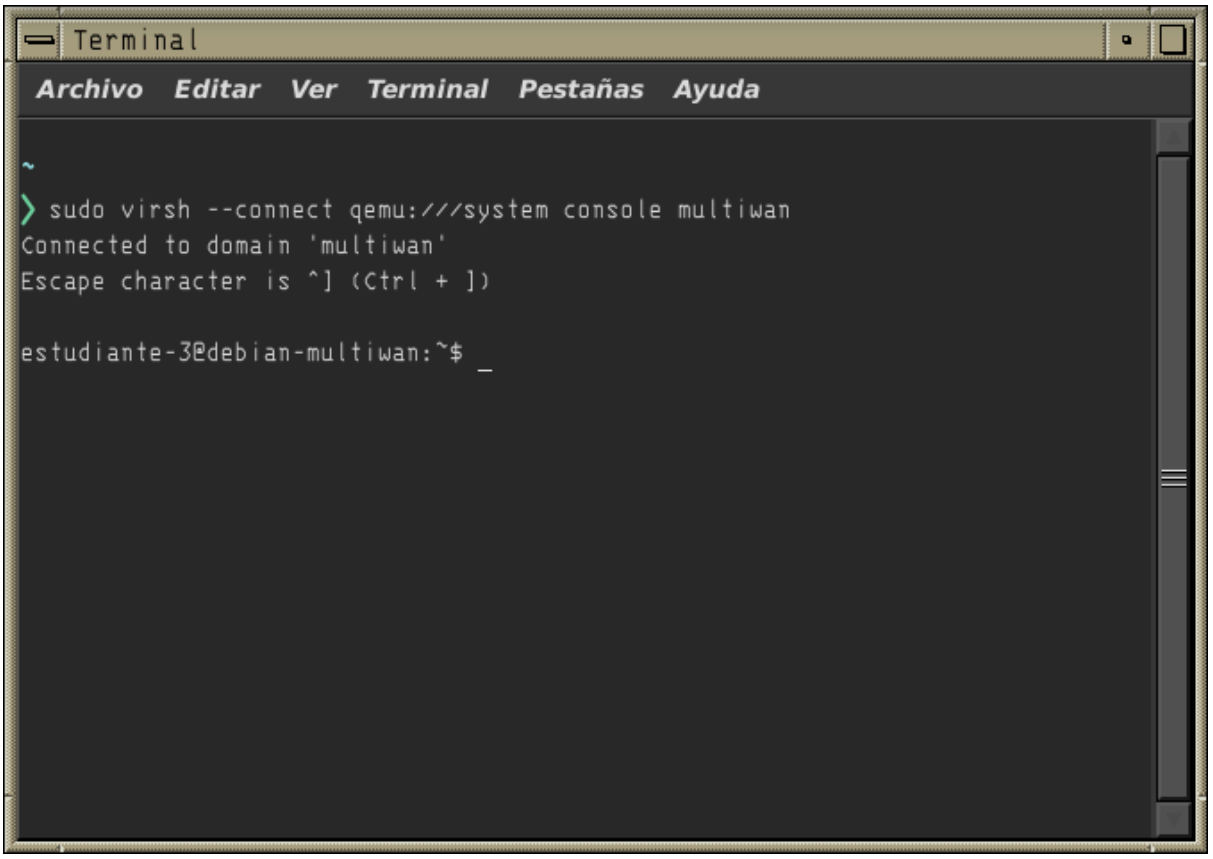
Comandos básicos de administración:

sudo virsh --connect qemu:///system start multiwan
sudo virsh --connect qemu:///system shutdown multiwan
sudo virsh --connect qemu:///system console multiwan
sudo virsh --connect qemu:///system list --all





Ingreso en la máquina virtual.



3. Asignación de dispositivos físicos a la VM

3.1 Teléfono Samsung — ISP1

Se creó `/tmp/multiwan-samsung.xml`:

```xml
<hostdev mode='subsystem' type='usb'>
  <source startupPolicy='optional'>
    <vendor id='0x04e8'/>
    <product id='0x6863'/>
  </source>
</hostdev>
```

Se añadió de forma persistente:

```bash
sudo virsh --connect qemu:///system attach-device \
  multiwan /tmp/multiwan-samsung.xml --config
```

3.2 Teléfono LG — ISP2

Se creó `/tmp/multiwan-lg.xml`:

```xml
<hostdev mode='subsystem' type='usb'>
  <source startupPolicy='optional' guestReset='off'>
    <vendor id='0x1004'/>
    <product id='0x6344'/>
  </source>
</hostdev>
```

Se añadió de forma persistente:

```bash
sudo virsh --connect qemu:///system attach-device \
  multiwan /tmp/multiwan-lg.xml --config
```

La opción `guestReset='off'` evita reinicios problemáticos del dispositivo LG por parte de la VM.

3.3 Adaptador Ethernet hacia el firewall

Se utilizó un adaptador Realtek `0bda:8152`. El archivo `/tmp/multiwan-ethernet.xml` quedó así:

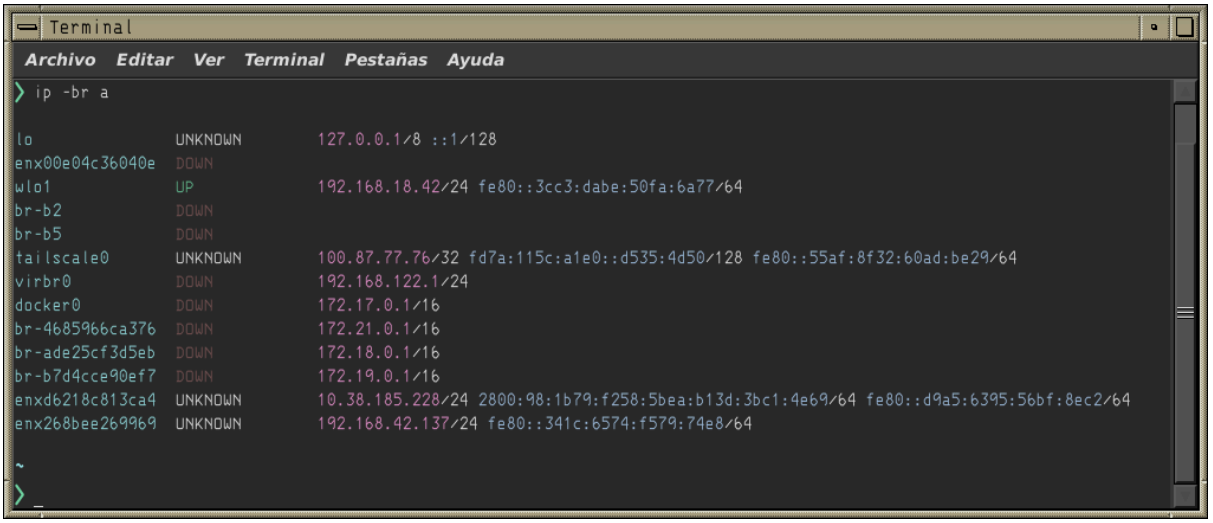
```xml
<hostdev mode='subsystem' type='usb'>
  <source startupPolicy='optional'>
    <vendor id='0x0bda'/>
    <product id='0x8152'/>
  </source>
</hostdev>
```

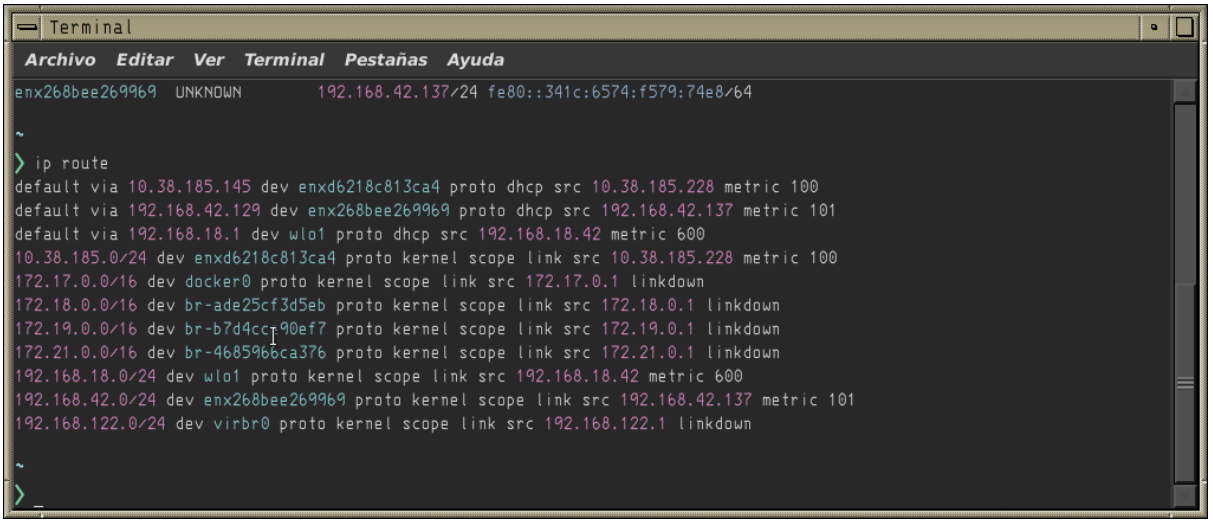
Se conectó en caliente y se guardó en la definición de la VM:

```bash
sudo virsh --connect qemu:///system attach-device \
  multiwan /tmp/multiwan-ethernet.xml --live --config
```

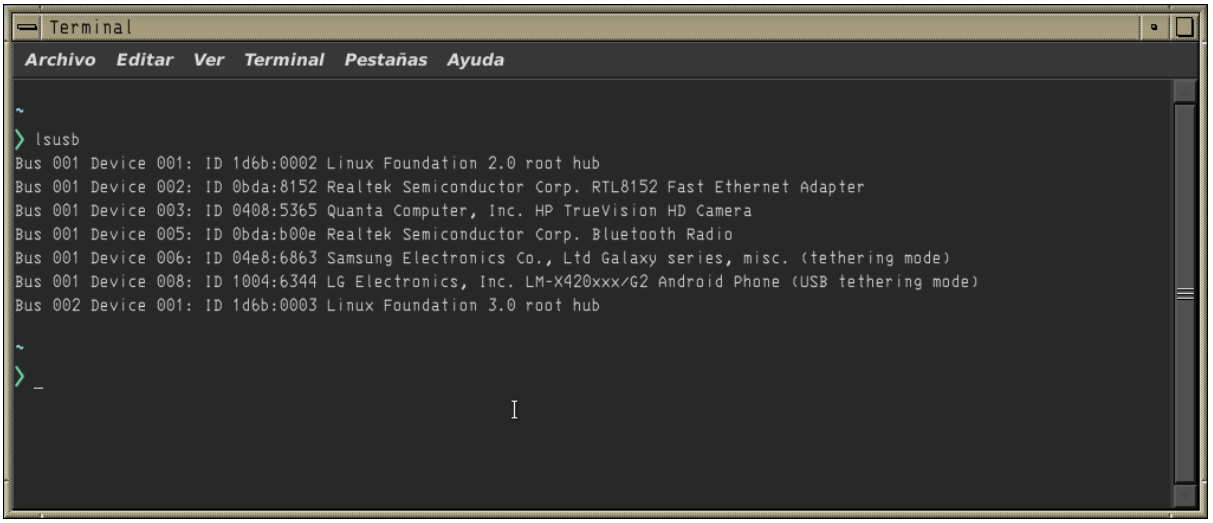
En la configuración final, la interfaz correspondiente dentro de R-EDGE es `enp7s0`.

Se conectan los dos dispositivos móviles por anclaje de red a la computadora anfitriona.

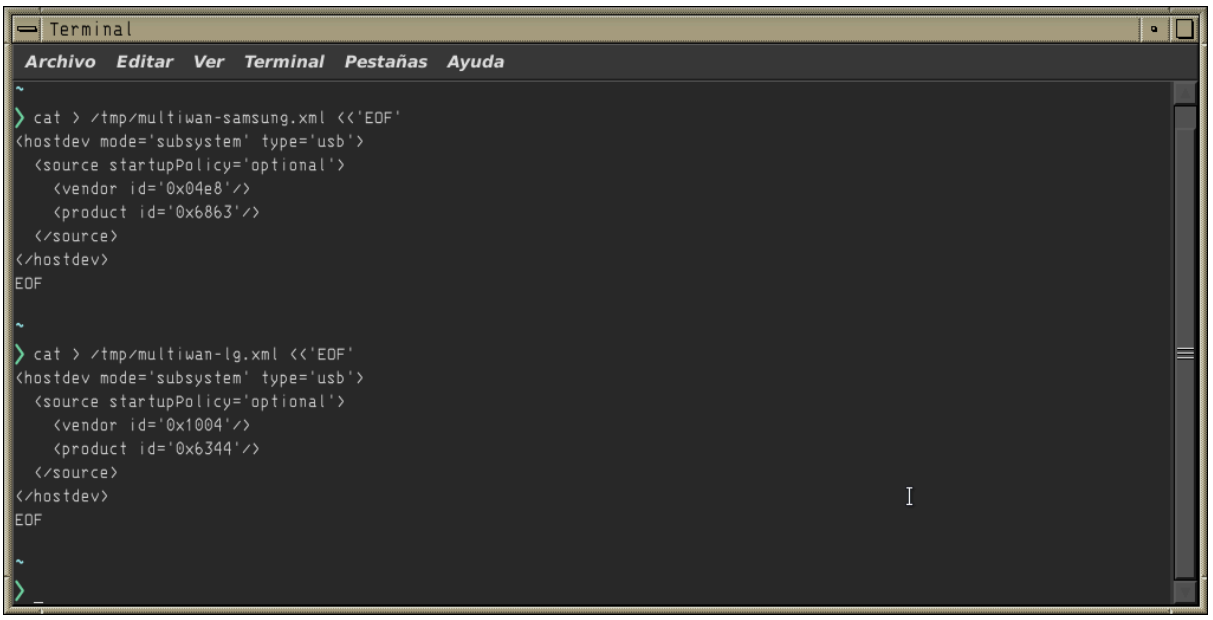




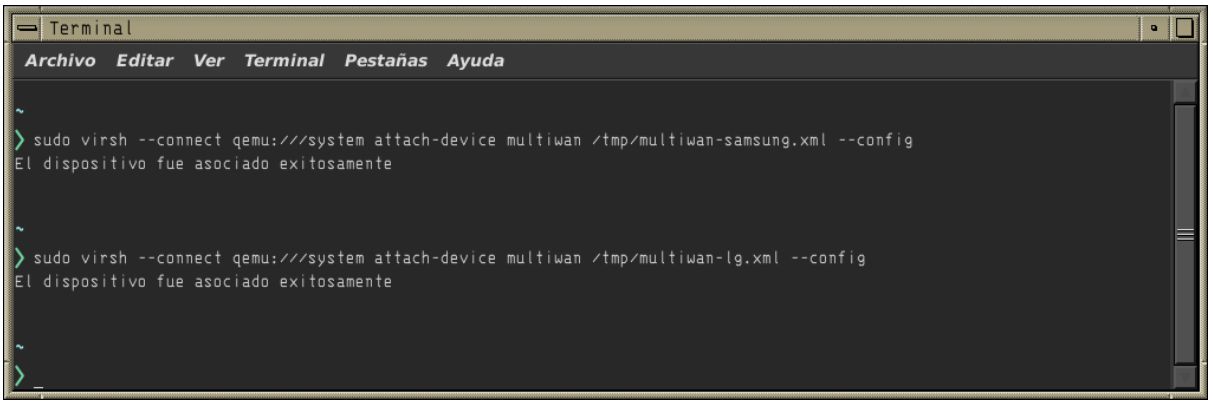
Dispositivos usb.



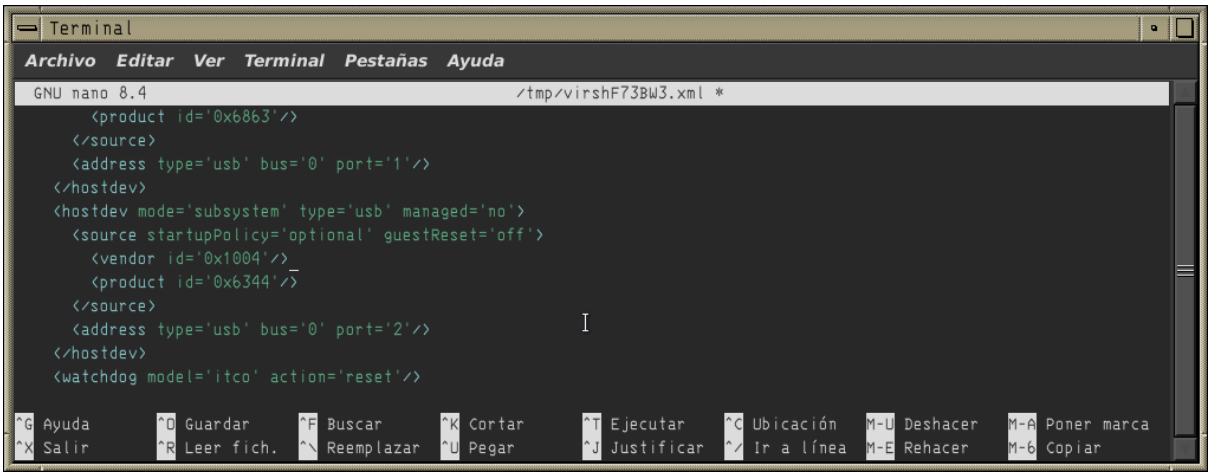
Creación de archivos para identificar cada de los teléfonos por fabricante y producto.
Optional permite el arranque aunque falte uno.



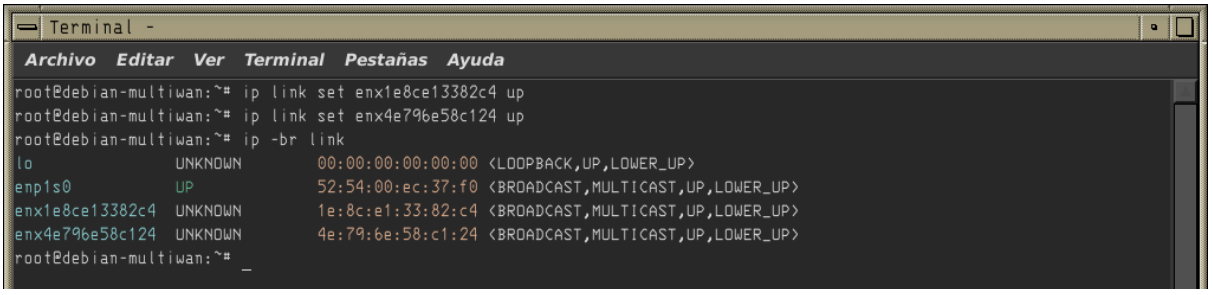
Asociar los dispositivos a la VM.



Cambiar la configuración de source para LG.



Activar interfaces.



4. Nombres persistentes de las WAN

Para identificar los teléfonos siempre como `wan1` y `wan2`, se creó `/etc/udev/rules.d/70-multiwan-net.rules`:

```udev
*SUBSYSTEM=="net", ACTION=="add", ATTRS{idVendor}=="04e8", ATTRS{idProduct}=="6863", NAME:="wan1"*
*SUBSYSTEM=="net", ACTION=="add", ATTRS{idVendor}=="1004", ATTRS{idProduct}=="6344", NAME:="wan2"*
```

Las reglas se recargaron con:

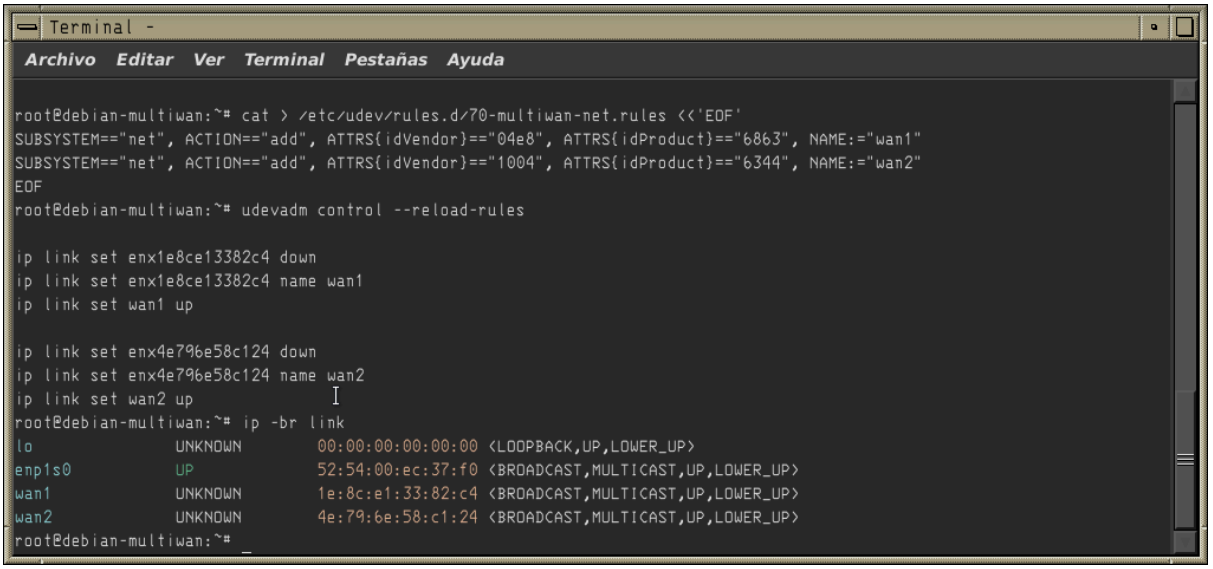
```bash
*sudo udevadm control --reload-rules*
```

Durante la configuración inicial se renombraron las interfaces detectadas:

```bash
*sudo ip link set enx1e8ce13382c4 down*
*sudo ip link set enx1e8ce13382c4 name wan1*
*sudo ip link set wan1 up*

*sudo ip link set enx4e796e58c124 down*
*sudo ip link set enx4e796e58c124 name wan2*
*sudo ip link set wan2 up*
```

Cambiar el nombre las interfaces.



5. Comprobación y asignación de direcciones WAN

Se instaló `arping`:

```bash
*sudo apt update*
*sudo apt install -y iputils-arping*
```

Antes de asignar las direcciones se comprobó que no estuvieran en uso:

```bash
```

sudo arping -D -I wan1 -c 3 10.38.185.228
sudo arping -D -I wan2 -c 3 192.168.42.137
```

Se comprobaron las puertas de enlace:

```bash
s*udo arping -I wan1 -c 3 10.38.185.145*
*sudo arping -I wan2 -c 3 192.168.42.129*
```
Pruebas de ips disponibles con arping.



6. Configuración persistente de red

6.1 Archivo principal

`/etc/network/interfaces`:

```ini
*source /etc/network/interfaces.d/**

*auto lo*
*iface lo inet loopback*

*# Interfaz de gestión de la VM, separada de la red del proyecto.*
*allow-hotplug enp1s0*
*iface enp1s0 inet dhcp*
```

6.2 Interfaces Multi-WAN

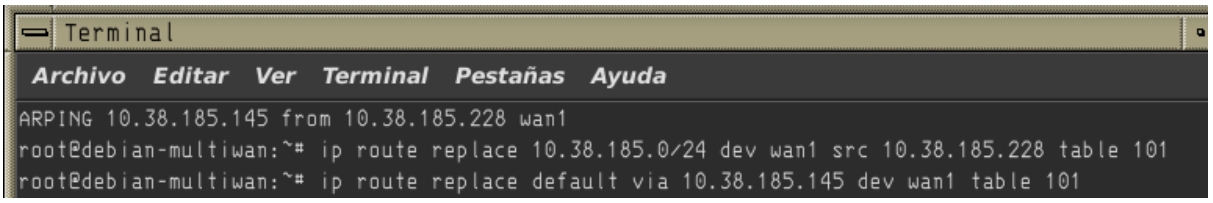
`/etc/network/interfaces.d/multiwan`:

```ini
*# WAN 1: Samsung*
*allow-hotplug wan1*
*iface wan1 inet static*
```
```

```
    address 10.38.185.228/24
    post-up ip route replace 10.38.185.0/24 dev wan1 src 10.38.185.228 table 101
    post-up ip route replace default via 10.38.185.145 dev wan1 table 101
    post-up ip rule del priority 1010 from 10.38.185.228/32 table 101 || true
    post-up ip rule add priority 1010 from 10.38.185.228/32 table 101
    pre-down ip rule del priority 1010 from 10.38.185.228/32 table 101 || true

# WAN 2: LG
allow-hotplug wan2
iface wan2 inet static
    address 192.168.42.137/24
    post-up ip route replace 192.168.42.0/24 dev wan2 src 192.168.42.137 table 102
    post-up ip route replace default via 192.168.42.129 dev wan2 table 102
    post-up ip rule del priority 1020 from 192.168.42.137/32 table 102 || true
    post-up ip rule add priority 1020 from 192.168.42.137/32 table 102
    pre-down ip rule del priority 1020 from 192.168.42.137/32 table 102 || true
```

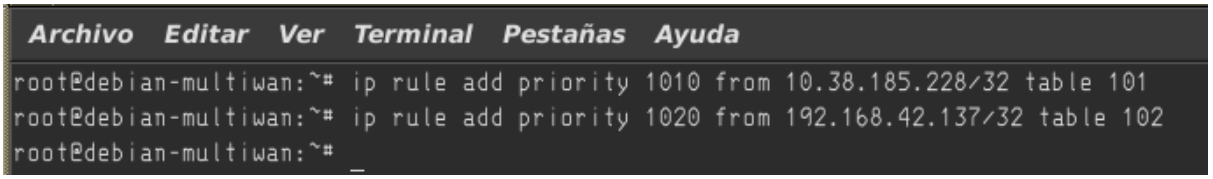
La tabla 101 contiene la ruta de ISP1 y la tabla 102 contiene la ruta de ISP2. Las reglas por dirección de origen garantizan que el tráfico propio de cada interfaz responda por el mismo enlace.
Configuración de la tabla de rutas 101 para el celular SAMSUNG.



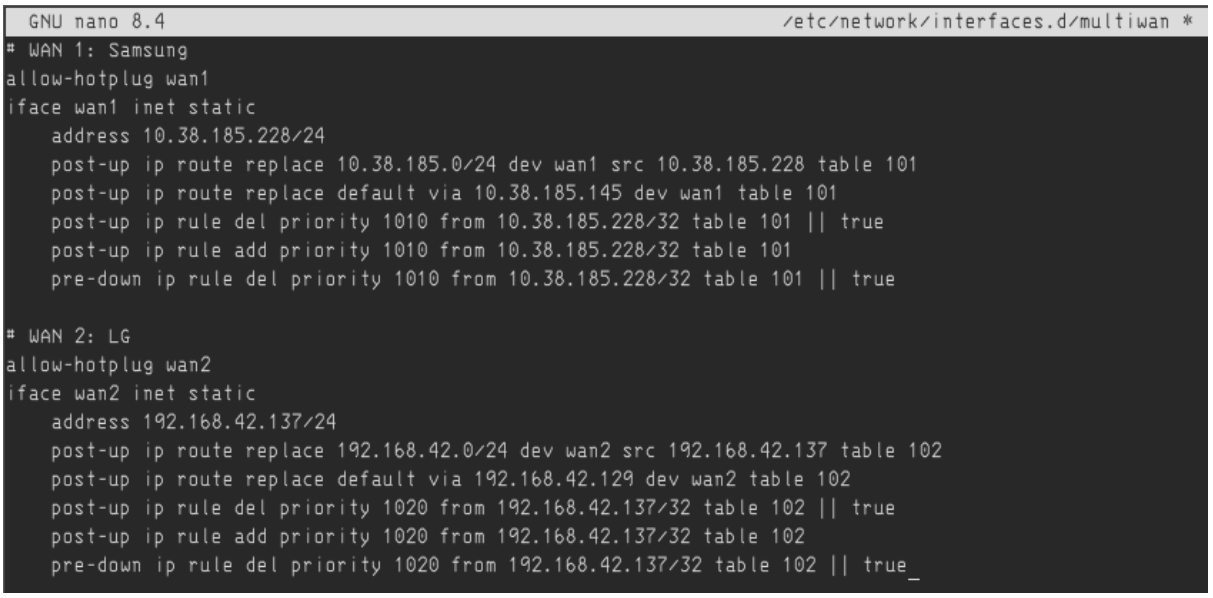
Configuración de la tabla de rutas 102 para el celular LG.



Agregar las reglas de selección a cada tabla.



Guardar las configuraciones de manera persistente.



6.3 Enlace de tránsito hacia el firewall

`/etc/network/interfaces.d/transito`:

```ini
# Enlace hacia el firewall
allow-hotplug enp7s0
iface enp7s0 inet static
    address 10.10.0.1/30

    # Rutas hacia las redes detrás del firewall
    post-up ip route replace 10.10.0.0/16 via 10.10.0.2 dev enp7s0
    post-up ip route replace 10.200.10.0/28 via 10.10.0.2 dev enp7s0
    post-up ip route replace 10.200.20.0/27 via 10.10.0.2 dev enp7s0

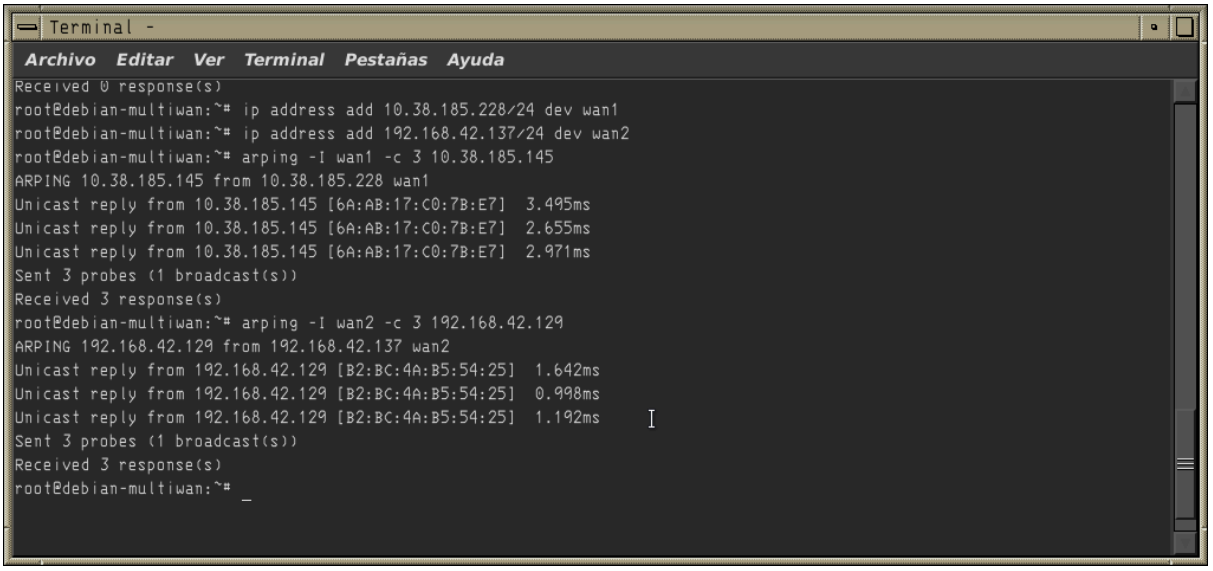
    # Los destinos internos se consultan en la tabla principal.
    post-up ip rule del priority 100 to 10.10.0.0/16 table main || true
    post-up ip rule add priority 100 to 10.10.0.0/16 table main
    post-up ip rule del priority 101 to 10.200.10.0/28 table main || true
    post-up ip rule add priority 101 to 10.200.10.0/28 table main
    post-up ip rule del priority 102 to 10.200.20.0/27 table main || true
    post-up ip rule add priority 102 to 10.200.20.0/27 table main

    pre-down ip rule del priority 100 to 10.10.0.0/16 table main || true
    pre-down ip rule del priority 101 to 10.200.10.0/28 table main || true
    pre-down ip rule del priority 102 to 10.200.20.0/27 table main || true

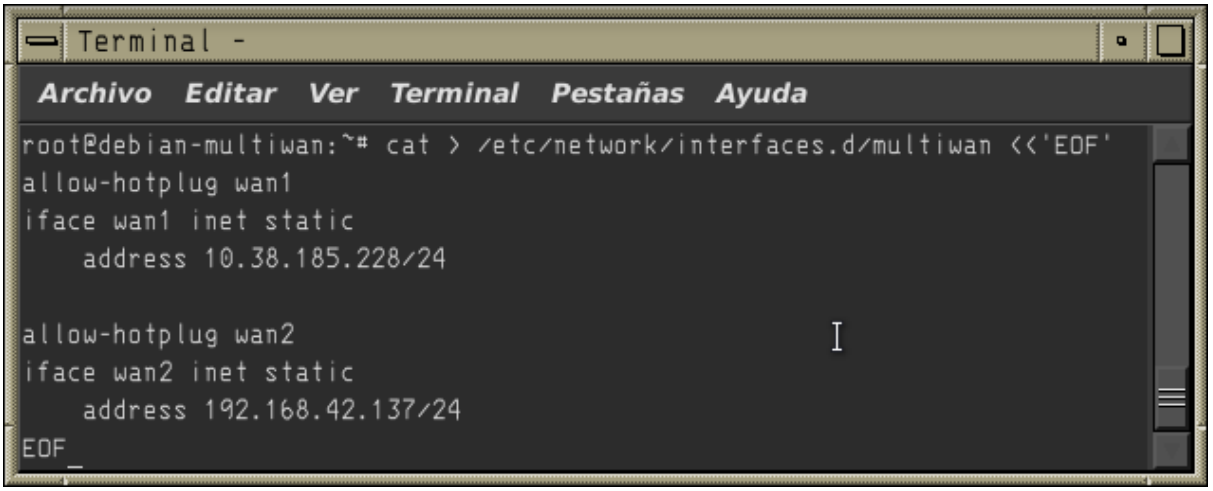
    # Ruta inicial para tráfico recibido desde el firewall.
```

```
    # Las marcas de prioridad 200 y 201 tienen preferencia sobre esta regla.
    post-up ip rule del priority 500 iif enp7s0 table 101 || true
    post-up ip rule add priority 500 iif enp7s0 table 101
    pre-down ip rule del priority 500 iif enp7s0 table 101 || true
```

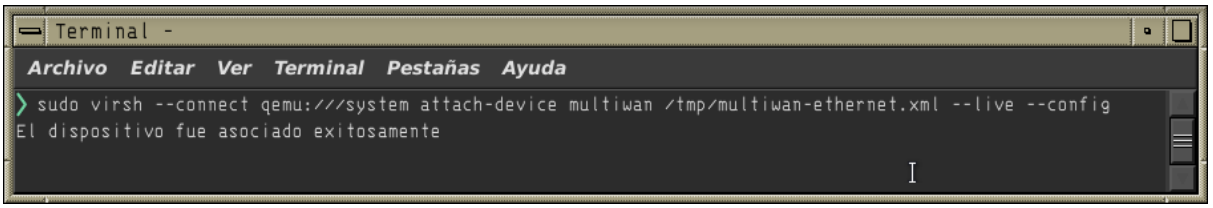
Asignación de ips temporales a las interfaces de los celulares.



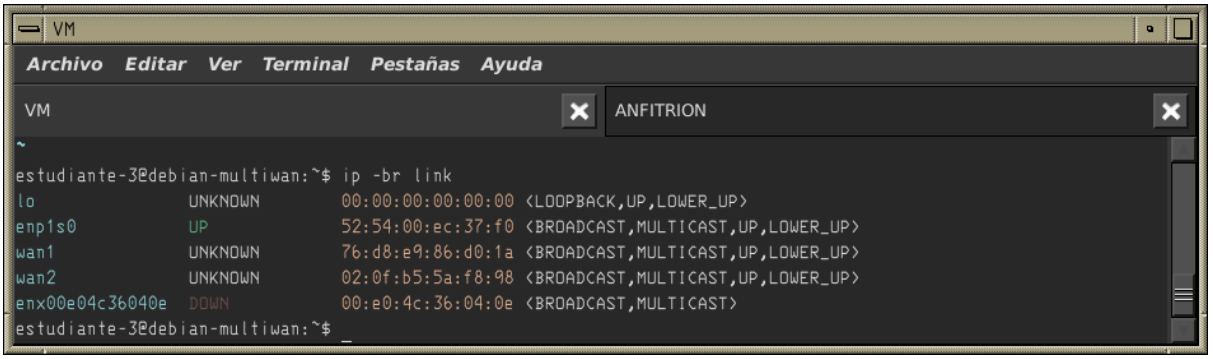
Asignación de ips a las wan.



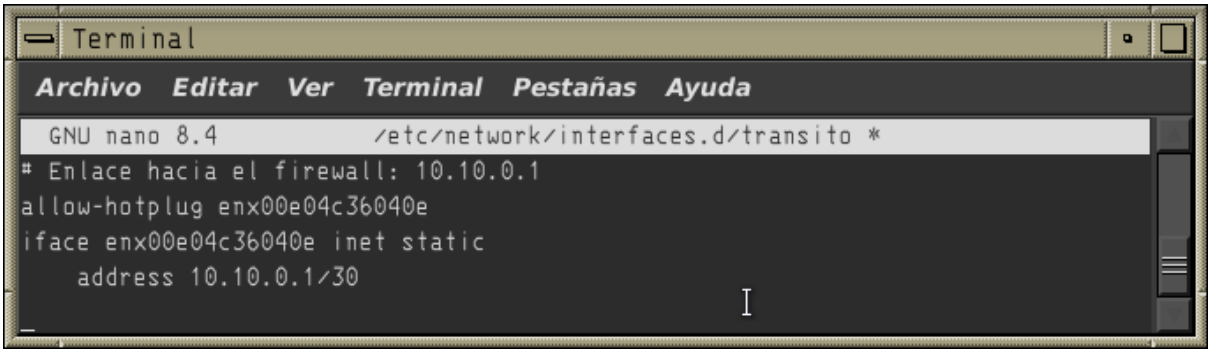
Asignar el dispositivo a la VM.



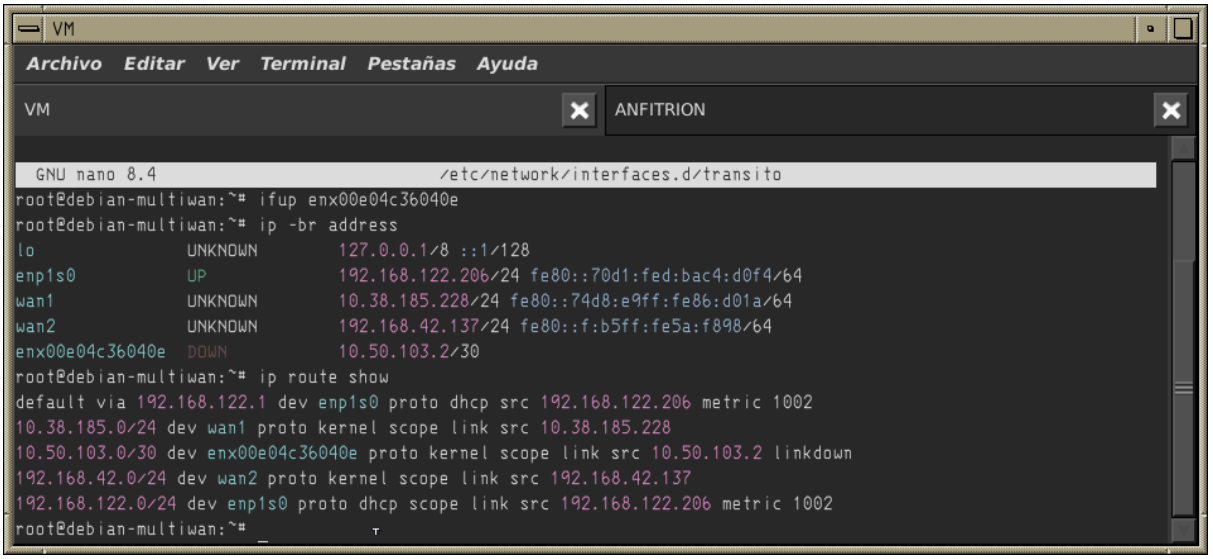
Comprobación.



Asignación de IPs estáticas.

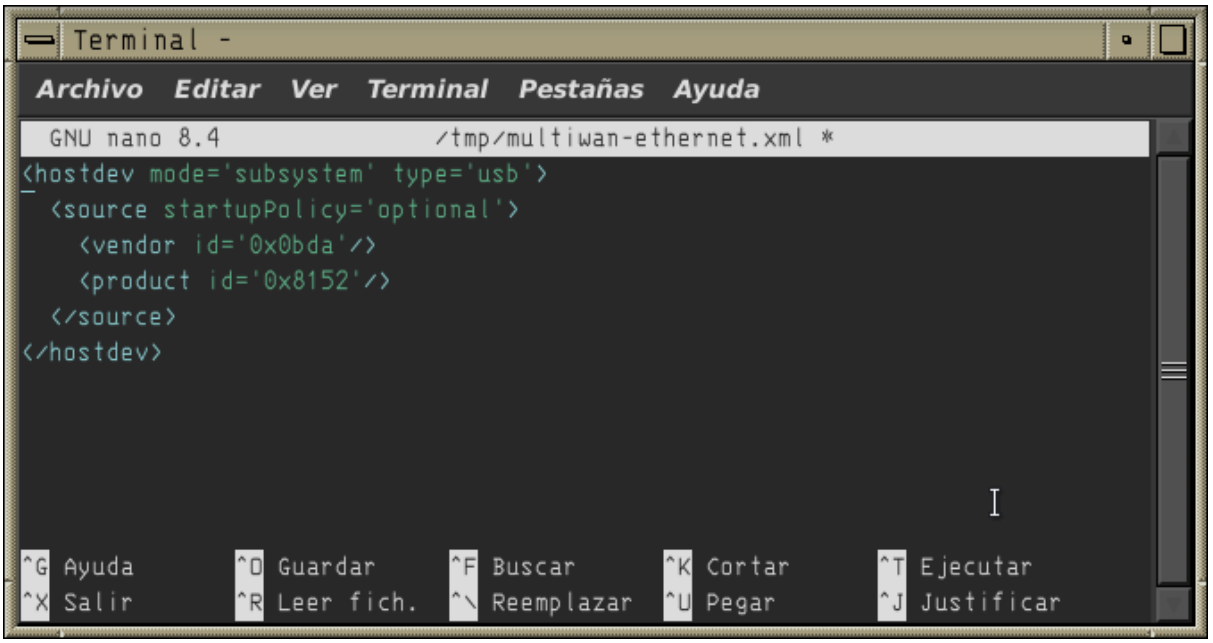


Activación de la interfaz.



Configuración de la interfaz que se conecta a Firewall.

Creación de archivo de asignación.



7. Reenvío IPv4

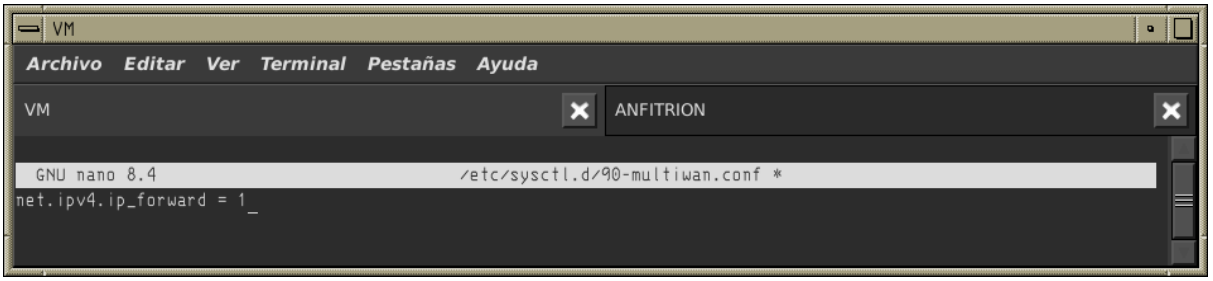
Los parámetros activos del núcleo son:

```ini
*net.ipv4.ip_forward = 1*
*net.ipv6.conf.all.disable_ipv6 = 1*
*net.ipv6.conf.default.disable_ipv6 = 1*
```

Se aplican con:

```bash
*sudo sysctl --system*
*sysctl net.ipv4.ip_forward*
```

Habilitación de forwarding ipv4.



8. Parámetros generales del Multi-WAN

Se creó el directorio de configuración:

```bash
sudo install -d -m 755 /etc/multiwan
```

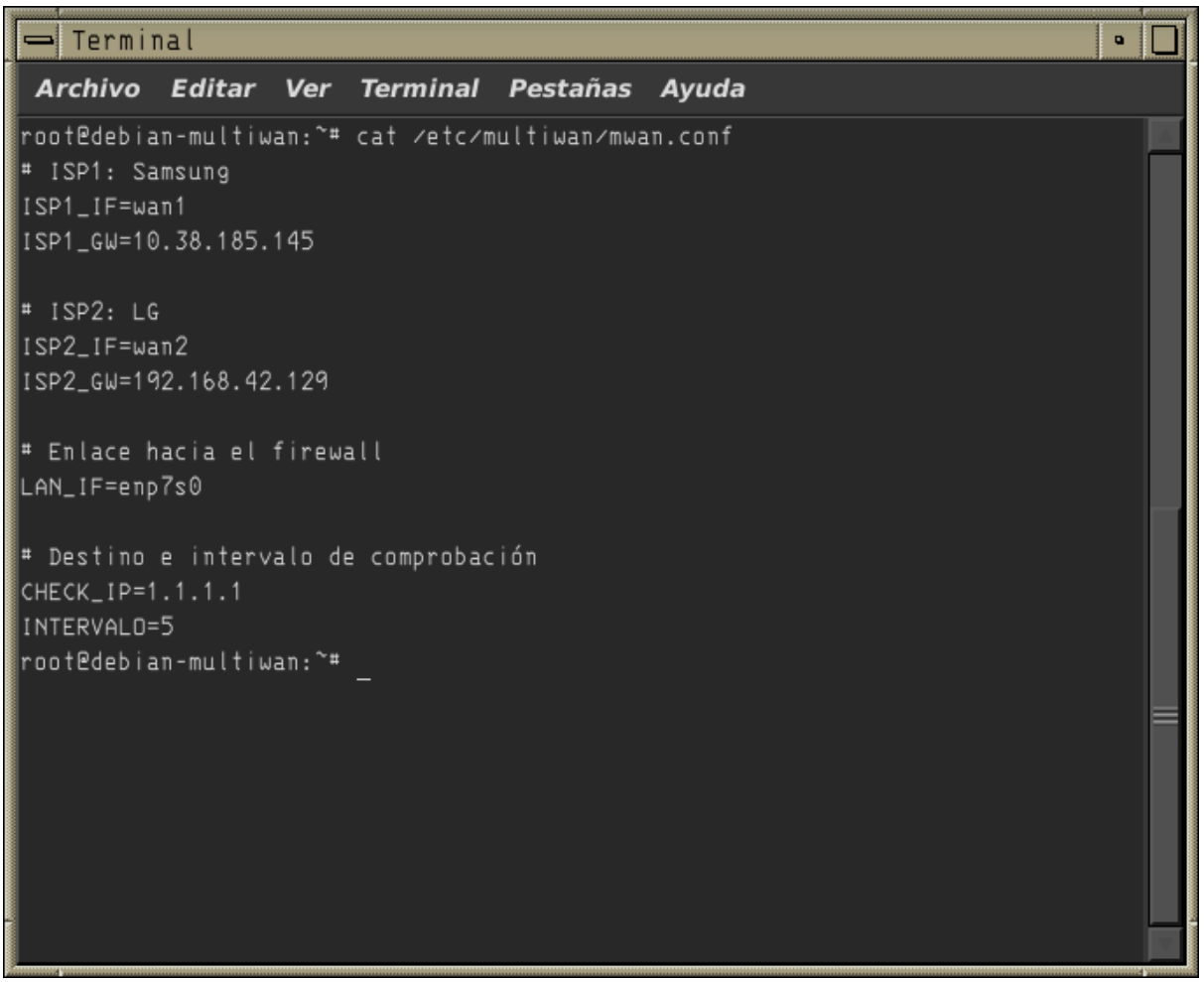
`/etc/multiwan/mwan.conf`:

```bash
# ISP1: Samsung
ISP1_IF=wan1
ISP1_GW=10.38.185.145

# ISP2: LG
ISP2_IF=wan2
ISP2_GW=192.168.42.129

# Enlace hacia el firewall
LAN_IF=enp7s0

# Destino e intervalo de comprobación
CHECK_IP=1.1.1.1
INTERVALO=5
```



9. Archivo externo de políticas

`/etc/multiwan/politicas.conf`:

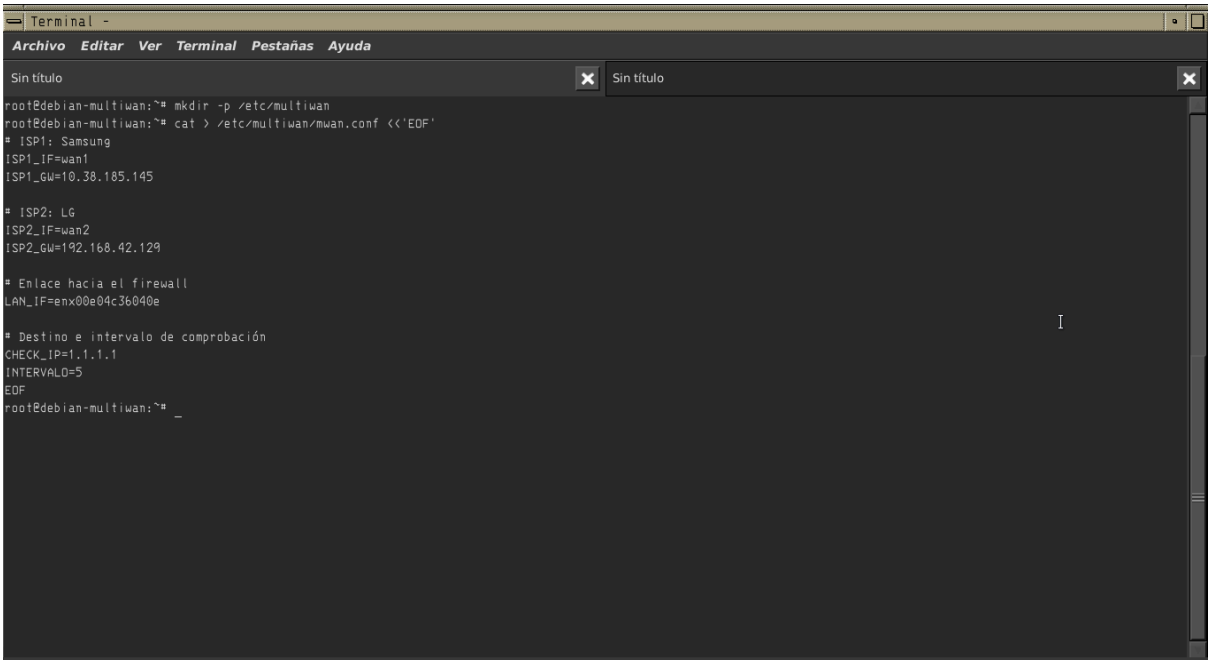
```text
# <ip_origen>, <puertos_destino>, <protocolo>, <ISP_salida>
10.10.20.10, [80,443], TCP, ISP1
10.10.20.10, [53], UDP, ISP2
10.10.20.11, [443], TCP, ISP1
10.10.10.0/27, [443], TCP, ISP1
10.10.40.0/27, [443], TCP, ISP2
10.10.10.10, [80], TCP, ISP2
10.10.0.6, [53], UDP, ISP2
```

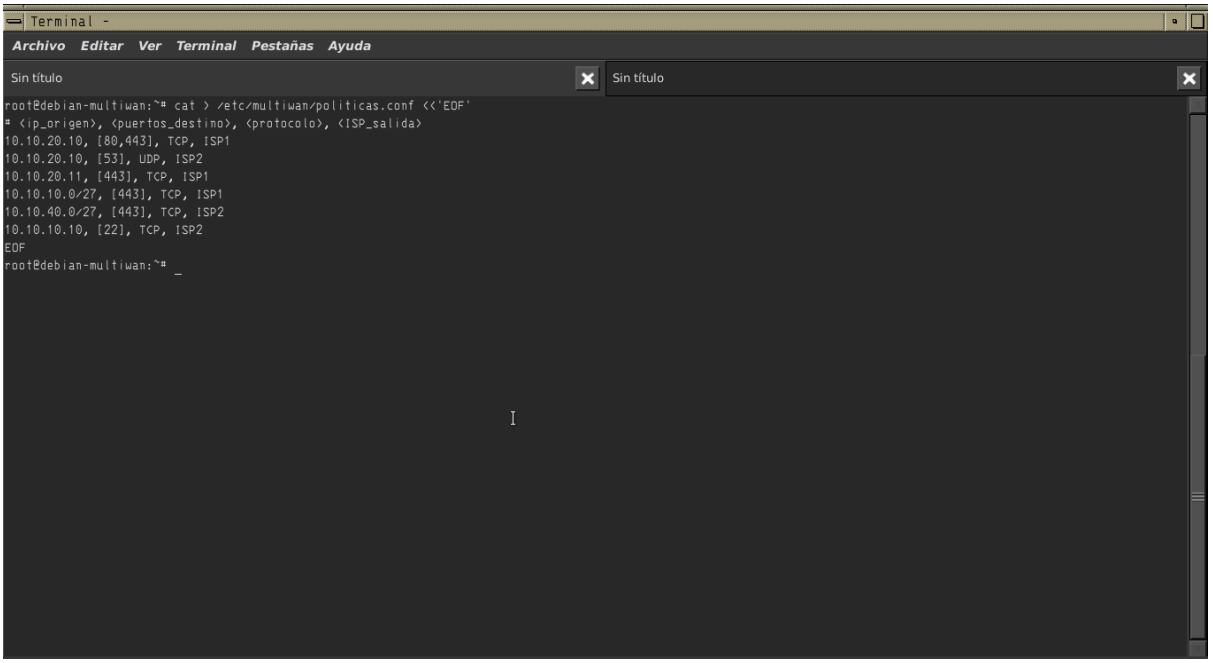
El archivo puede modificarse sin cambiar el código del sistema. Después de cada cambio se debe ejecutar:

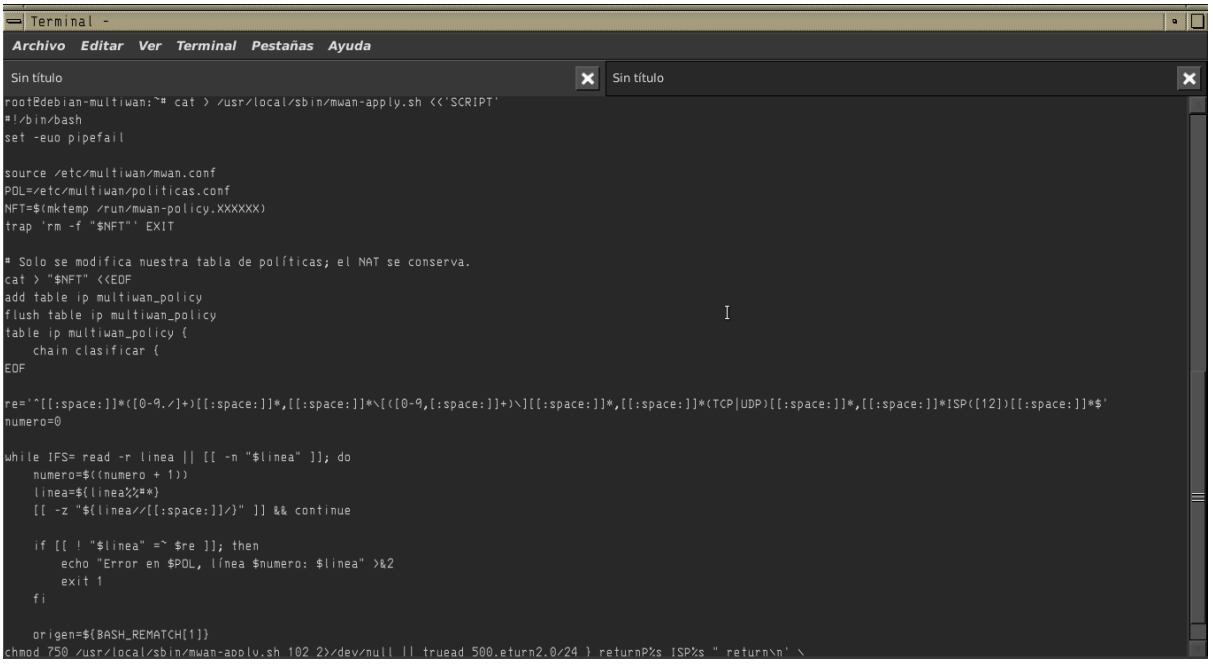
```bash
sudo /usr/local/sbin/mwan-apply.sh
```

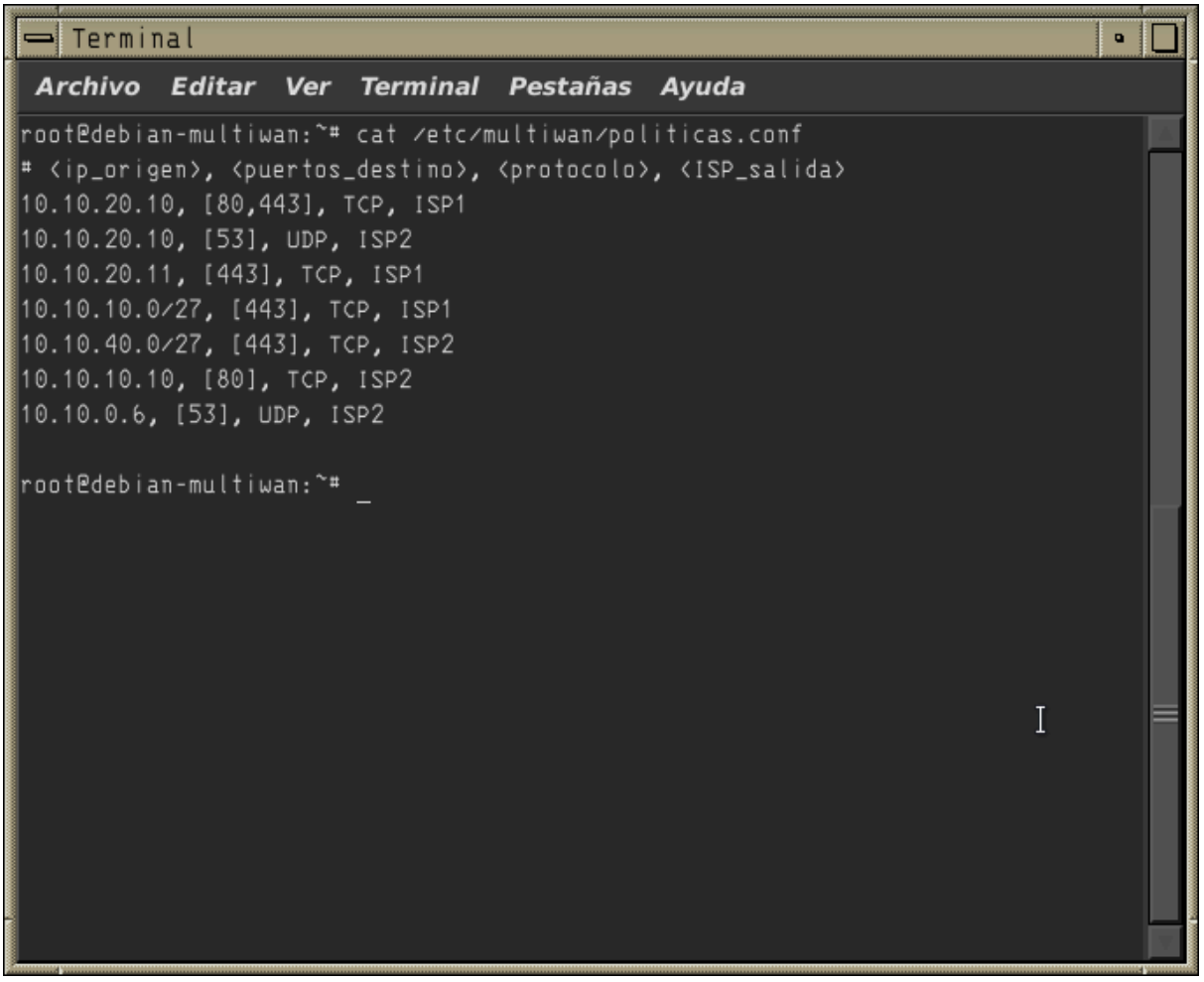
```

El tráfico web conserva la dirección del cliente después de pasar por el proxy. Las consultas DNS son resueltas por la caché del proxy y salen con origen `10.10.0.6`; por eso tienen una política específica.





```





10. NAT y reenvío de WireGuard

`/etc/nftables.conf`:

```nft
#!/usr/sbin/nft -f

flush ruleset

table ip multiwan_nat {
   set redes_proyecto {
      type ipv4_addr
      flags interval
      elements = { 10.10.0.0/16, 10.200.10.0/28, 10.200.20.0/27 }
   }

   chain prerouting {
      type nat hook prerouting priority dstnat; policy accept;

      iifname "wan1" fib daddr type local udp dport 51820 counter log prefix "MWAN-VPN
ISP1 " dnat to 10.10.50.2:51820
      iifname "wan2" fib daddr type local udp dport 51820 counter log prefix "MWAN-VPN
ISP2 " dnat to 10.10.50.2:51820
   }

   chain postrouting {
      type nat hook postrouting priority 100; policy accept;

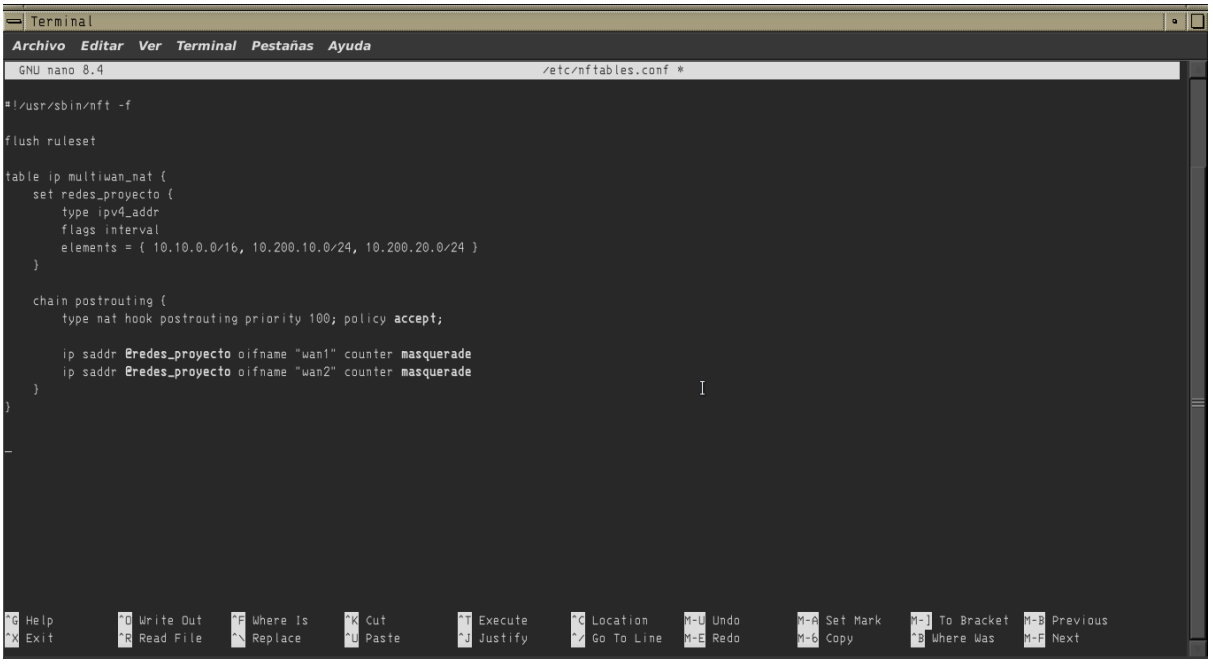
      ip saddr @redes_proyecto oifname "wan1" counter masquerade
      ip saddr @redes_proyecto oifname "wan2" counter masquerade
   }
}

include "/etc/multiwan/playit-egress.nft"
```

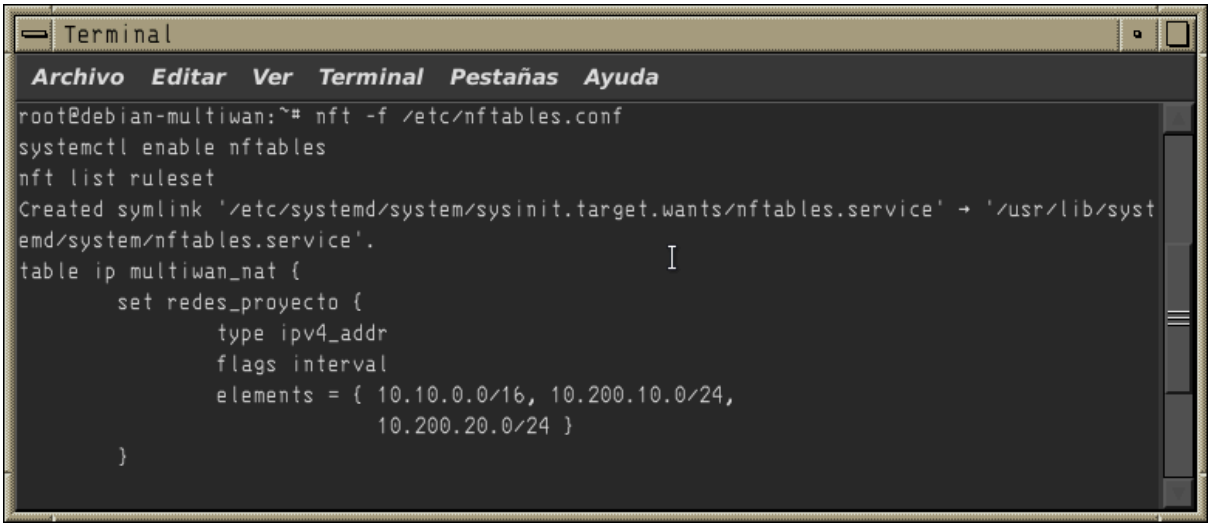
Se validó y habilitó con:

```bash
sudo nft -c -f /etc/nftables.conf
sudo systemctl enable --now nftables
sudo nft list table ip multiwan_nat
```

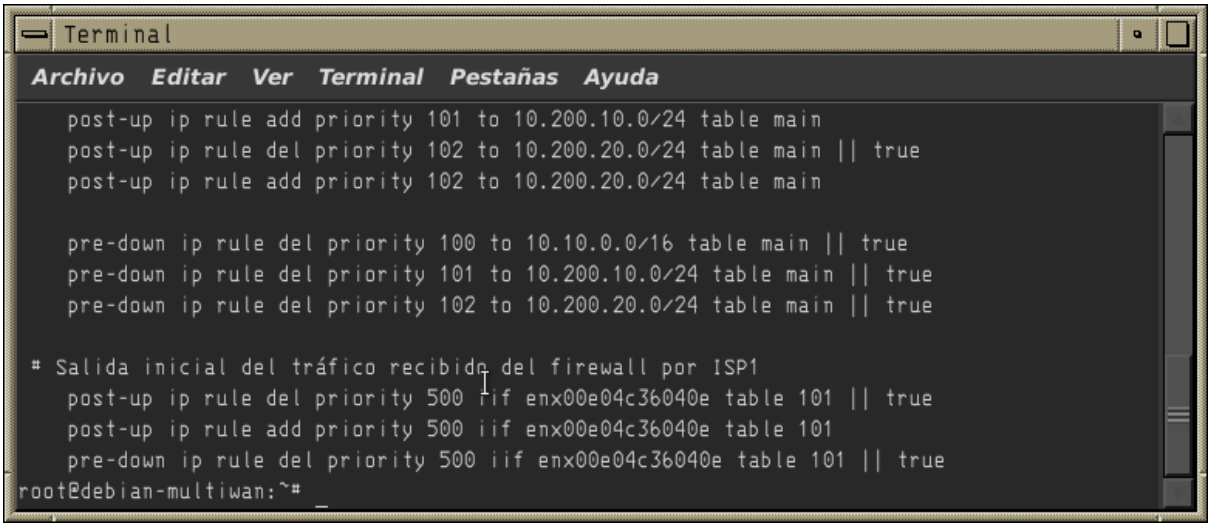
Configuración de reglas de NAT.



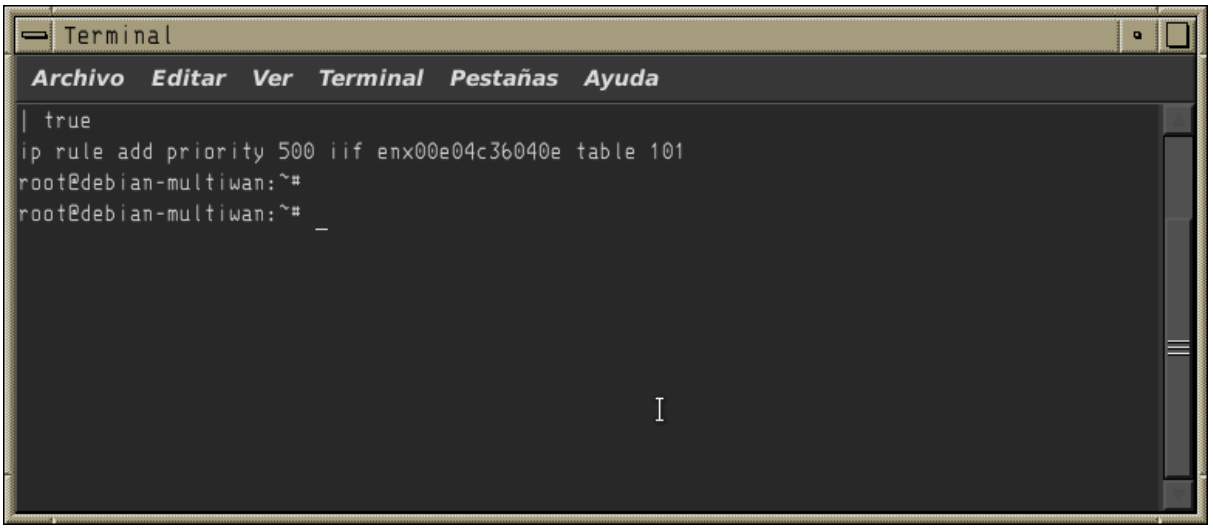
Aplicación y habilitación.



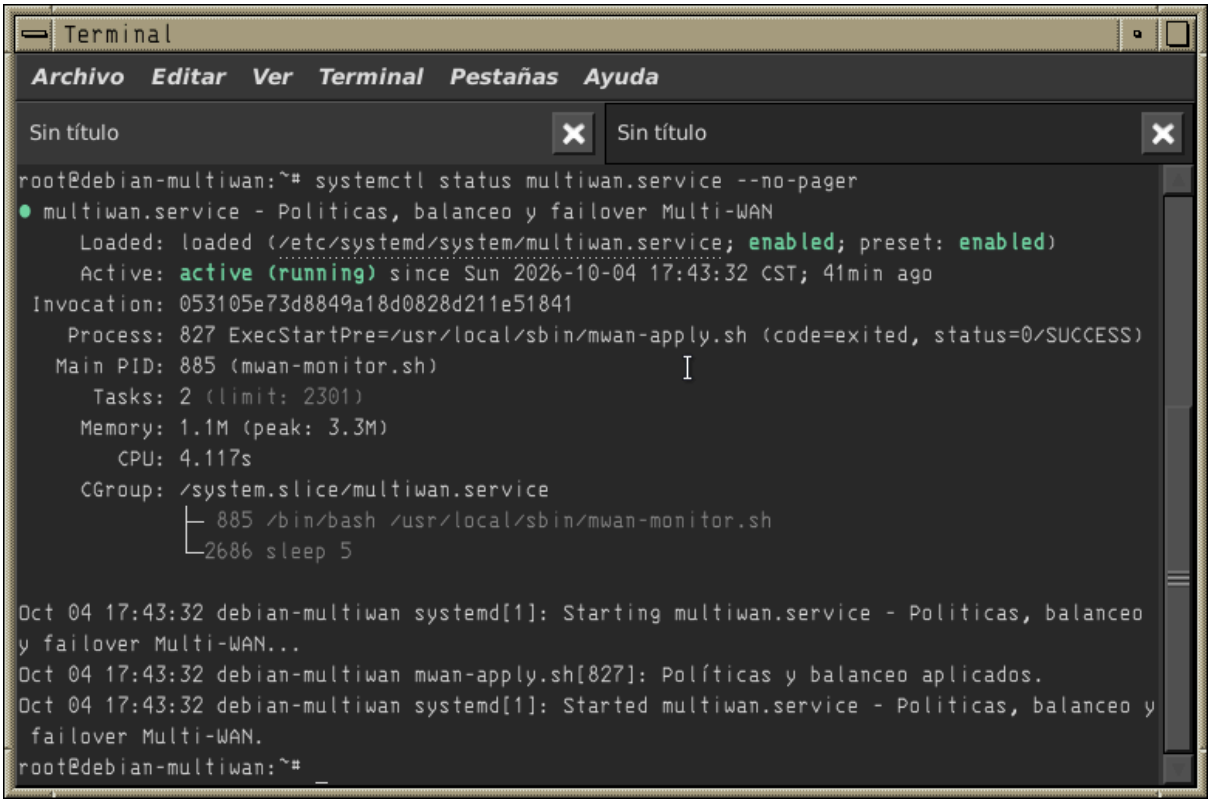
Reglas de salida



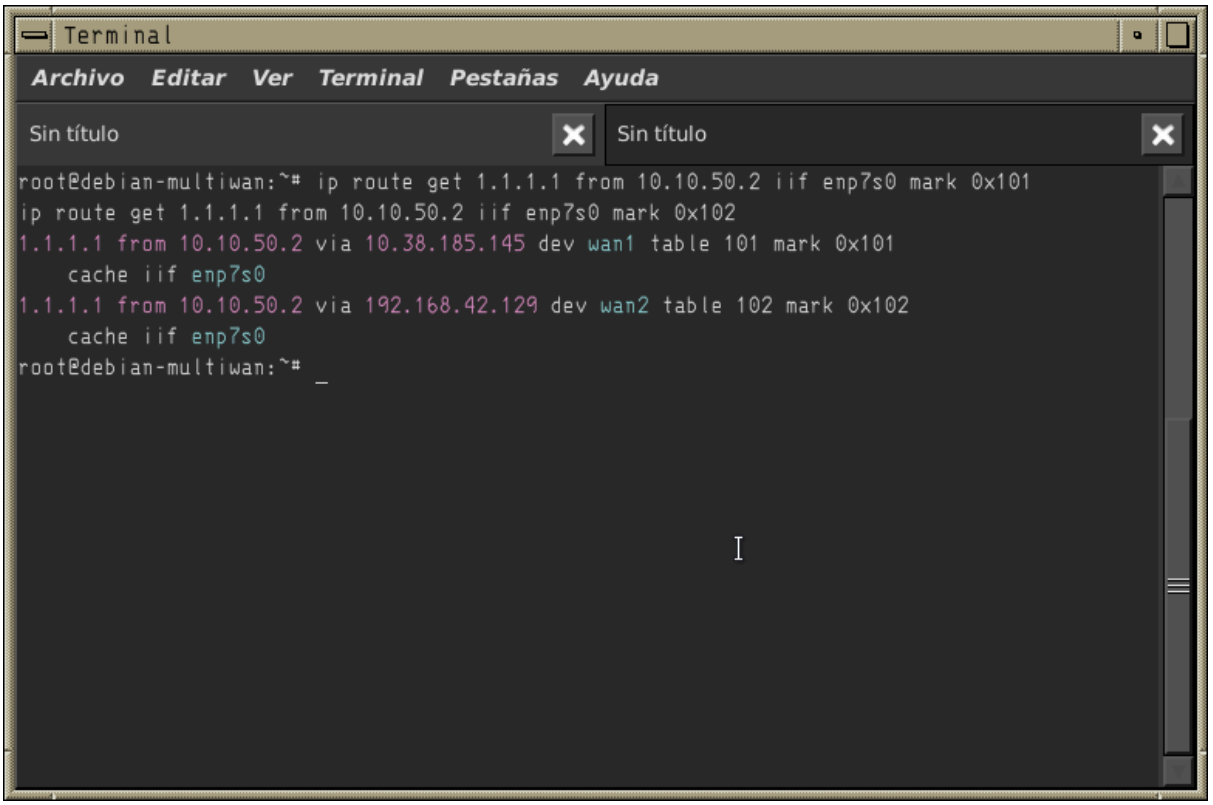
aplicación



Configuración de entrada wireguard.
Servicio multiwan activo



Comprobaciones.



11. Aplicación de políticas y balanceo

`/usr/local/sbin/mwan-apply.sh`:

```bash
#!/bin/bash
set -euo pipefail

source /etc/multiwan/mwan.conf
POL=/etc/multiwan/politicas.conf
NFT=$(mktemp /run/mwan-policy.XXXXXX)
trap 'rm -f "$NFT"' EXIT

# Solo se modifica la tabla de políticas; la tabla de NAT se conserva.
cat > "$NFT" <<EOF
add table ip multiwan_policy
flush table ip multiwan_policy
table ip multiwan_policy {
    chain clasificar {
EOF

re='^[[:space:]]*([0-9./]+)[[:space:]]*,[[:space:]]*
                                        ([0 − 9, [: space:]] +)
```

```
[[:space:]]*,[[:space:]]*(TCP|UDP)[[:space:]]*,[[:space:]]*ISP([12])[[:space:]]*$'
numero=0

while IFS= read -r linea || [[ -n "$linea" ]]; do
    numero=$((numero + 1))
    linea=${linea%%#*}
    [[ -z "${linea//[[:space:]]/}" ]] && continue

    if [[ ! "$linea" =~ $re ]]; then
        echo "Error en $POL, línea $numero: $linea" >&2
        exit 1
    fi

    origen=${BASH_REMATCH[1]}
    puertos=${BASH_REMATCH[2]//[[:space:]]/}
    protocolo=${BASH_REMATCH[3],,}
    isp=${BASH_REMATCH[4]}

    printf '        ip saddr %s %s dport { %s } counter meta mark set %s log prefix "MWAN-P%s
ISP%s " return\n' \
        "origen""protocolo" "puertos""isp" "numero""isp" >> "$NFT"
done < "$POL"

cat >> "$NFT" <<EOF
        # Sin política: elegir un ISP para esta conexión.
        meta mark set numgen random mod 2 offset 1
        meta mark 1 counter log prefix "MWAN-BAL ISP1 "
        meta mark 2 counter log prefix "MWAN-BAL ISP2 "
    }

    chain registrar_salida {
        type filter hook postrouting priority mangle; policy accept;

        iifname != "$LAN_IF" return
        ct state != new return
        ct direction != original return

        ct mark 1 oifname "$ISP1_IF" counter log prefix "MWAN-REAL M1 ISP1 "
        ct mark 1 oifname "$ISP2_IF" counter log prefix "MWAN-REAL M1 ISP2 "
        ct mark 2 oifname "$ISP1_IF" counter log prefix "MWAN-REAL M2 ISP1 "
        ct mark 2 oifname "$ISP2_IF" counter log prefix "MWAN-REAL M2 ISP2 "
    }

    chain entrada {
        type filter hook prerouting priority mangle; policy accept;

        # Guardar el ISP por el que entra WireGuard.
        ct state invalid,untracked return
```

```
        iifname "$ISP1_IF" fib daddr type local udp dport 51820 ct state new ct mark set 0x101
return
        iifname "$ISP2_IF" fib daddr type local udp dport 51820 ct state new ct mark set 0x102
return

        # Clasificar el tráfico recibido desde el firewall.
        iifname != "$LAN_IF" return

        # No aplicar salida WAN a destinos internos o de gestión.
        ip daddr { 10.10.0.0/16, 10.200.10.0/28, 10.200.20.0/27, 192.168.122.0/24 } return
        ip saddr != { 10.10.0.0/16, 10.200.10.0/28, 10.200.20.0/27 } return
        ct state invalid,untracked return

        # Recuperar la elección de conexiones clasificadas previamente.
        ct mark != 0 meta mark set ct mark return

        # Clasificar conexiones nuevas y guardar la elección.
        ct state new jump clasificar
        meta mark != 0 ct mark set meta mark
    }
}
EOF

# Validar antes de cargar.
nft -c -f "$NFT"
nft -f "$NFT"

# Las marcas se consultan antes de la regla inicial de prioridad 500.
while ip rule del priority 200 fwmark 1 2>/dev/null; do
    :
done
ip rule add priority 200 fwmark 1 table 101

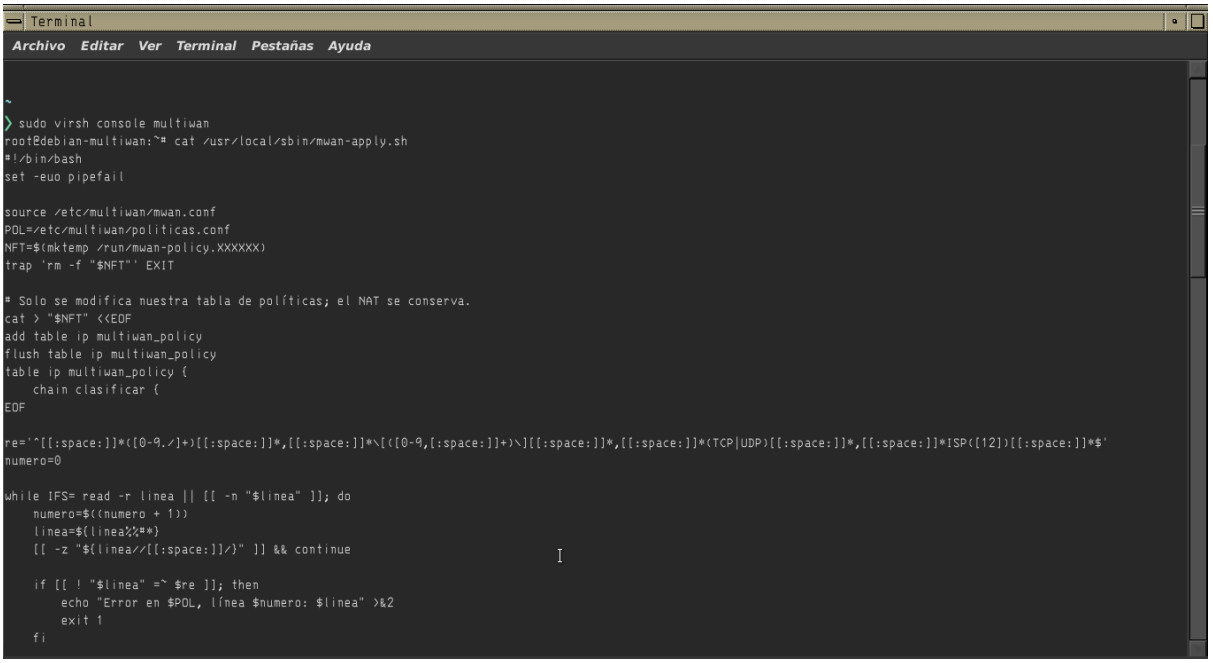
while ip rule del priority 201 fwmark 2 2>/dev/null; do
    :
done
ip rule add priority 201 fwmark 2 table 102

# Las respuestas de WireGuard regresan por el ISP de entrada.
while ip rule del priority 120 fwmark 0x101 2>/dev/null; do
    :
done
ip rule add priority 120 fwmark 0x101 table 101

while ip rule del priority 121 fwmark 0x102 2>/dev/null; do
    :
done
ip rule add priority 121 fwmark 0x102 table 102
```

echo "Políticas y balanceo aplicados."
```

El balanceo se realiza por conexión. Cuando no existe una política específica, `numgen random mod 2 offset 1` asigna la marca 1 o 2 con una probabilidad del 50 %. La marca se guarda en `ct mark` para que todos los paquetes de la misma conexión utilicen el mismo proveedor.




# 12. Monitor y failover automático

`/usr/local/sbin/mwan-monitor.sh`:

```bash
*#!/bin/bash*
*set -euo pipefail*
*source /etc/multiwan/mwan.conf*

*ip rule del priority 110 fwmark 0x11 table 101 2>/dev/null || true*
*ip rule add priority 110 fwmark 0x11 table 101*

*ip rule del priority 111 fwmark 0x12 table 102 2>/dev/null || true*
*ip rule add priority 111 fwmark 0x12 table 102*

*ip route replace unreachable default table 103*

*sondeo() {*
```
```

```
    if ping -4 -n -I "1" − m"2" -c 2 -W 2 \
        "$CHECK_IP" >/dev/null 2>&1; then
        echo up
    else
        echo down
    fi
}

aplicar() {
    local estado1="1"estado2 = "2"
    local tabla1 tabla2

    case "estado1:estado2" in
        up:up)     tabla1=101; tabla2=102 ;;
        up:down)   tabla1=101; tabla2=101 ;;
        down:up)   tabla1=102; tabla2=102 ;;
        down:down) tabla1=103; tabla2=103 ;;
    esac

    ip rule del priority 200 fwmark 1 2>/dev/null || true
    ip rule add priority 200 fwmark 1 table "$tabla1"

    ip rule del priority 201 fwmark 2 2>/dev/null || true
    ip rule add priority 201 fwmark 2 table "$tabla2"

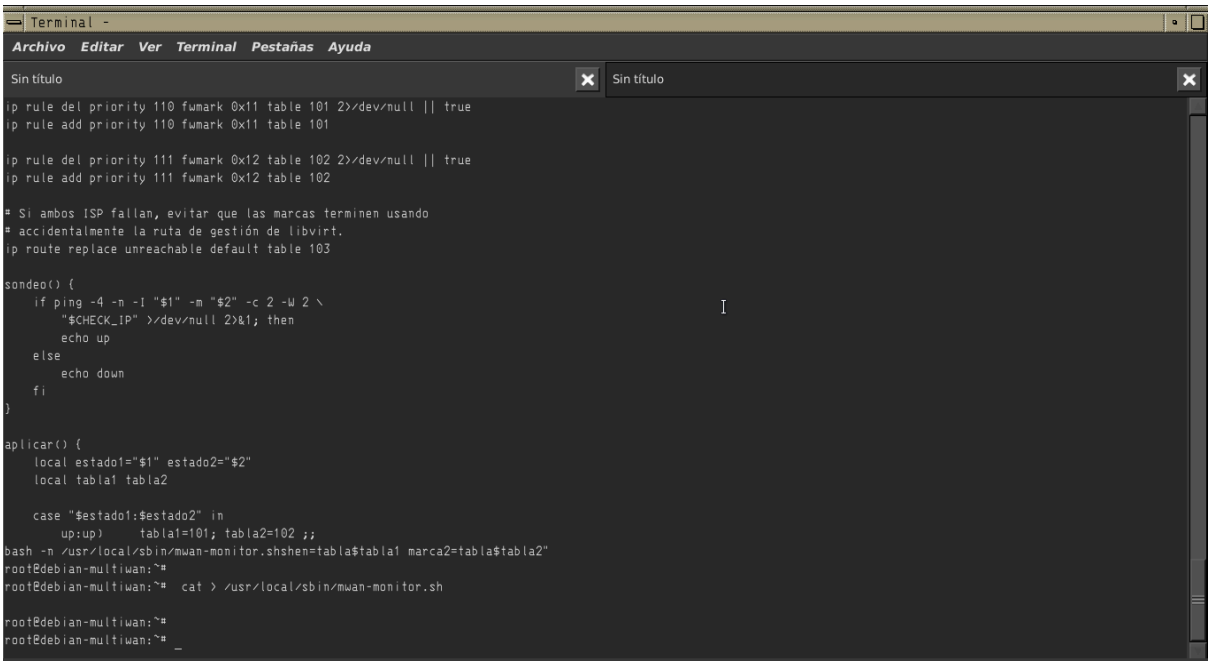
    logger -t multiwan \
        "ISP1=estado1ISP2 =estado2 marca1=tablatabla1marca2 = tablatabla2"
}

anterior=""
while true; do
    estado1=(sondeo"ISP1_IF" 17)
    estado2=(sondeo"ISP2_IF" 18)
    actual="estado1:estado2"

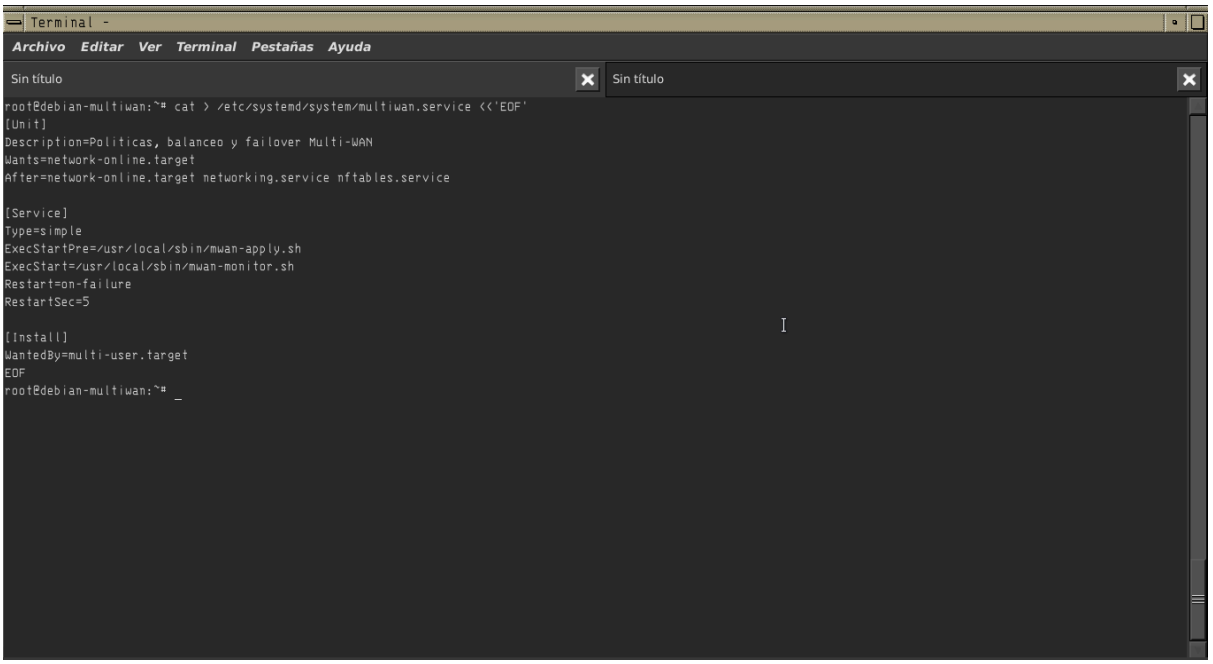
    if [[ "actual"! = "anterior" ]]; then
        aplicar "estado1""estado2"
        anterior="$actual"
    fi

    sleep "$INTERVALO"
done
```

Creación de monitor para el failover.



Creación de servicio de arranque.



Estado ISP1	Estado ISP2	Marca 1	Marca 2
Activo	Activo	Tabla 101	Tabla 102
Activo	Caído	Tabla 101	Tabla 101
Caído	Activo	Tabla 102	Tabla 102
Caído	Caído	Tabla 103, inalcanzable	Tabla 103, inalcanzable

13. Servicio persistente de Multi-WAN

`/etc/systemd/system/multiwan.service`:

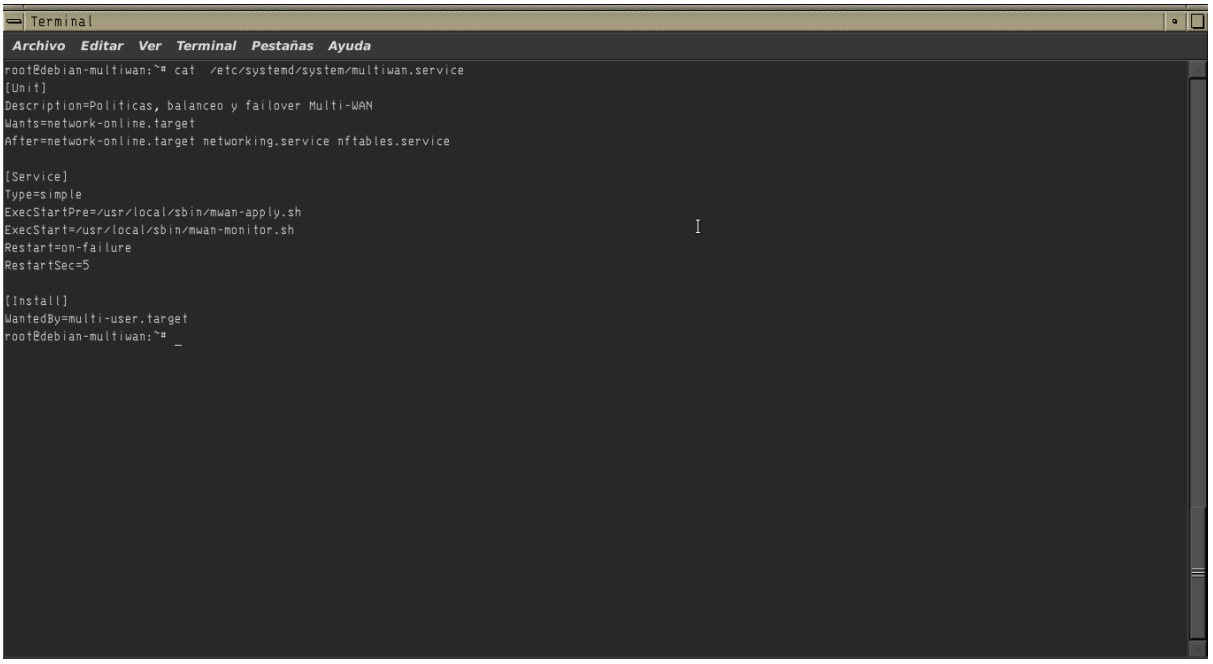
```ini
*[Unit]*
*Description=Politicas, balanceo y failover Multi-WAN*
*Wants=network-online.target*
*After=network-online.target networking.service nftables.service*

*[Service]*
*Type=simple*
*ExecStartPre=/usr/local/sbin/mwan-apply.sh*
*ExecStart=/usr/local/sbin/mwan-monitor.sh*
*Restart=on-failure*
*RestartSec=5*

*[Install]*
*WantedBy=multi-user.target*
```

Se instalaron permisos y se habilitó el servicio:

```bash
*sudo chmod 750 /usr/local/sbin/mwan-apply.sh*
*sudo chmod 750 /usr/local/sbin/mwan-monitor.sh*
*sudo systemctl daemon-reload*
*sudo systemctl enable --now multiwan.service*
```



14. Salida del agente de playit por el Multi-WAN

El agente proporciona el punto de entrada público utilizado por la VPN. Para que su tráfico propio no utilice la interfaz de gestión, se creó `/etc/multiwan/playit-egress.nft`:

```nft
table ip playit_egress {
    chain salida {
        type route hook output priority mangle; policy accept;

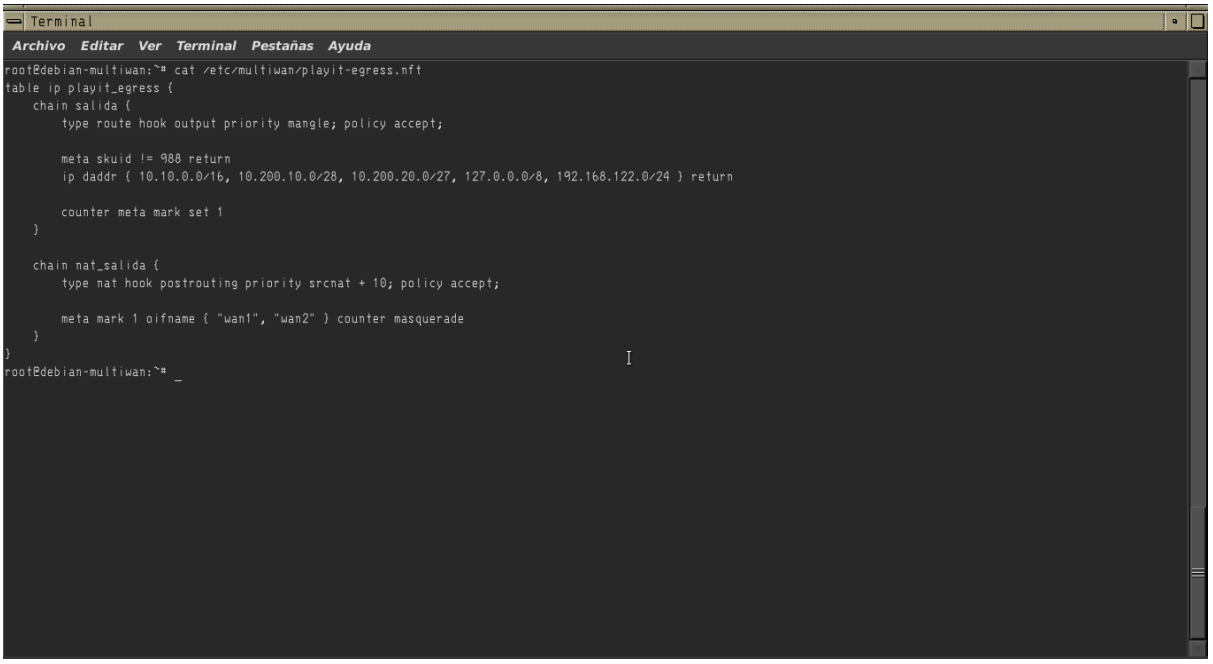
        meta skuid != 988 return
        ip daddr { 10.10.0.0/16, 10.200.10.0/28, 10.200.20.0/27, 127.0.0.0/8, 192.168.122.0/24
} return

        counter meta mark set 1
    }

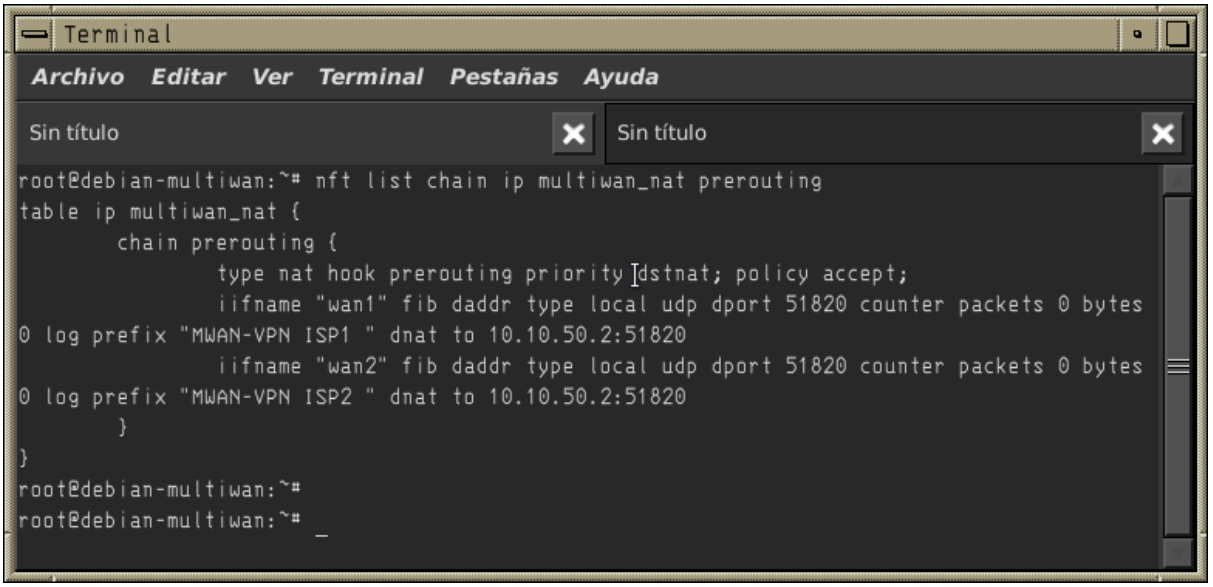
    chain nat_salida {
        type nat hook postrouting priority srcnat + 10; policy accept;

        meta mark 1 oifname { "wan1", "wan2" } counter masquerade
    }
}
```

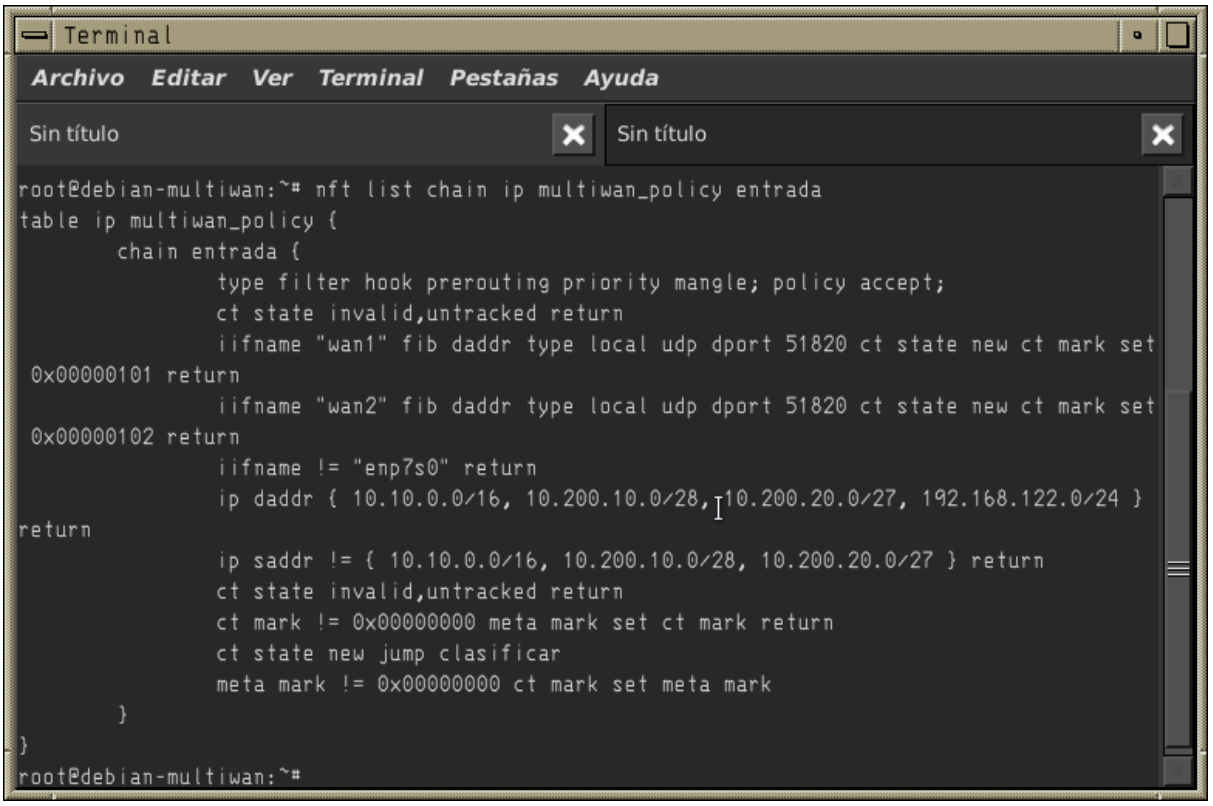
La regla selecciona el proceso mediante el UID del usuario del servicio, aplica la marca 1 y permite que el monitor lo redirija al ISP disponible si ISP1 falla.



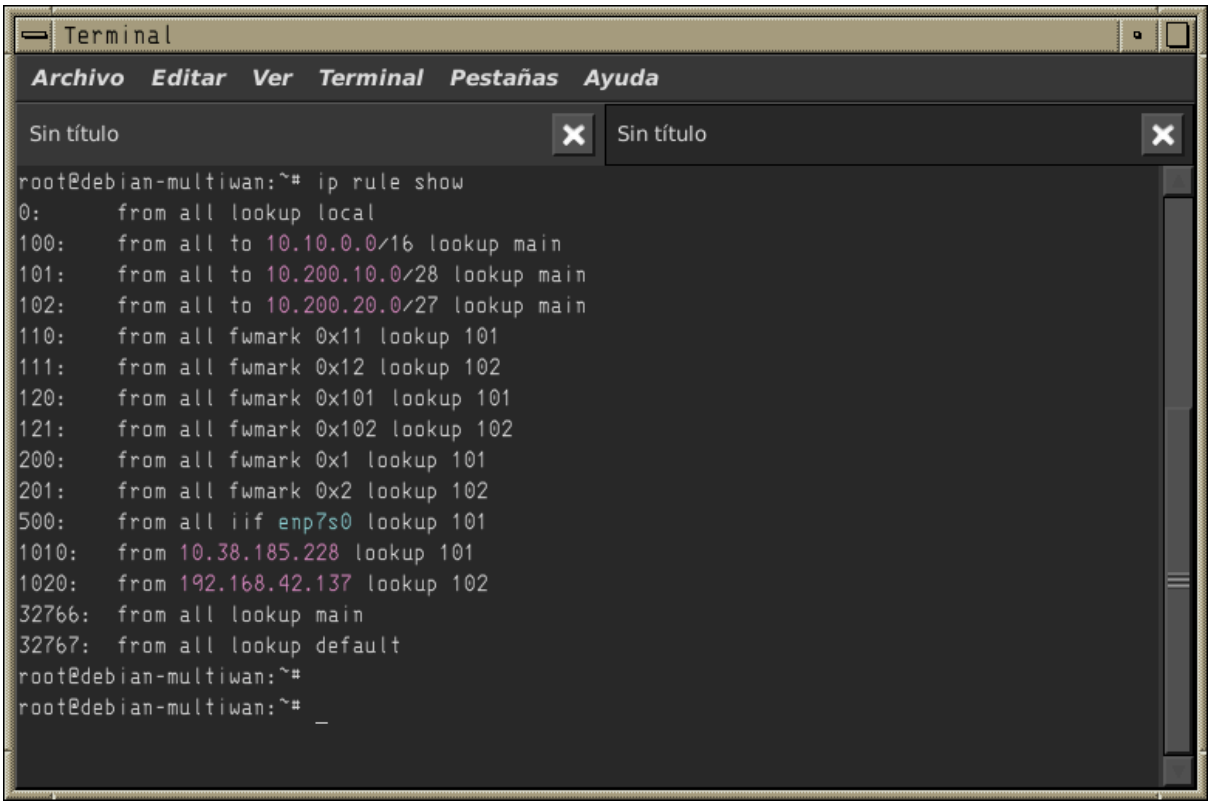
Reenvio UDP al servidor VPN.



Marcado según el ISP de entrada.



Reglas de retorno ISP



15. Agente de Zabbix

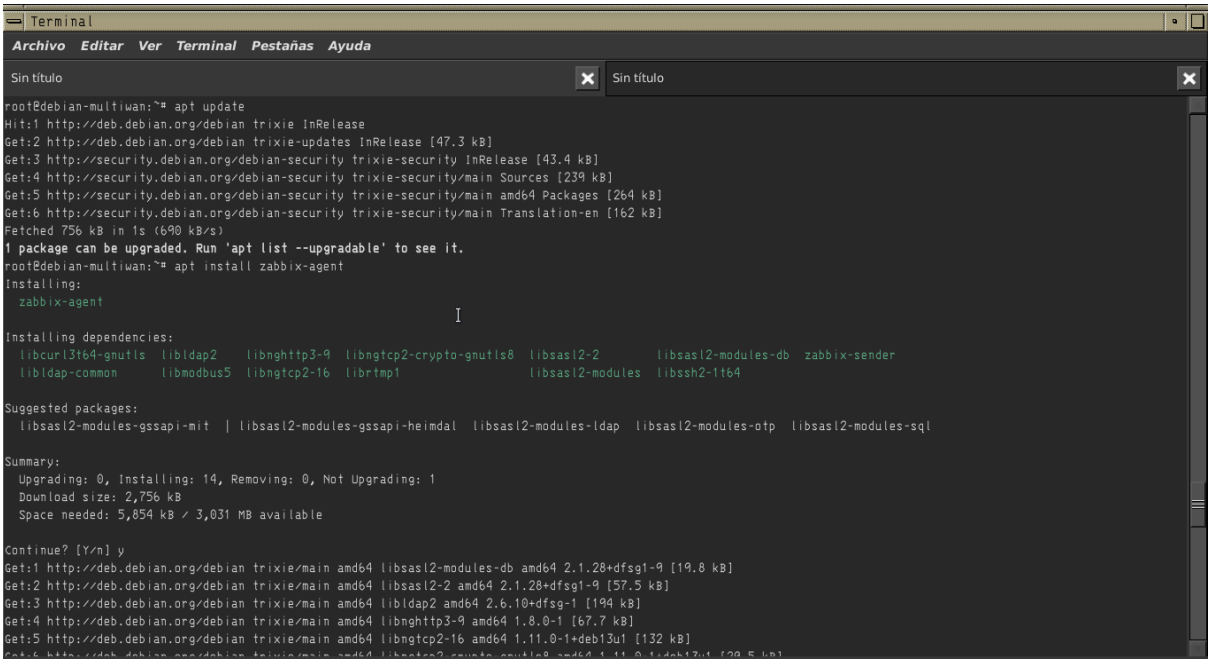
Los parámetros principales de `/etc/zabbix/zabbix_agentd.conf` son:

```ini
*PidFile=/run/zabbix/zabbix_agentd.pid*
*LogFile=/var/log/zabbix-agent/zabbix_agentd.log*
*LogFileSize=0*
*Server=10.10.30.2*
*ServerActive=10.10.30.2*
*Include=/etc/zabbix/zabbix_agentd.conf.d/*.conf*
```

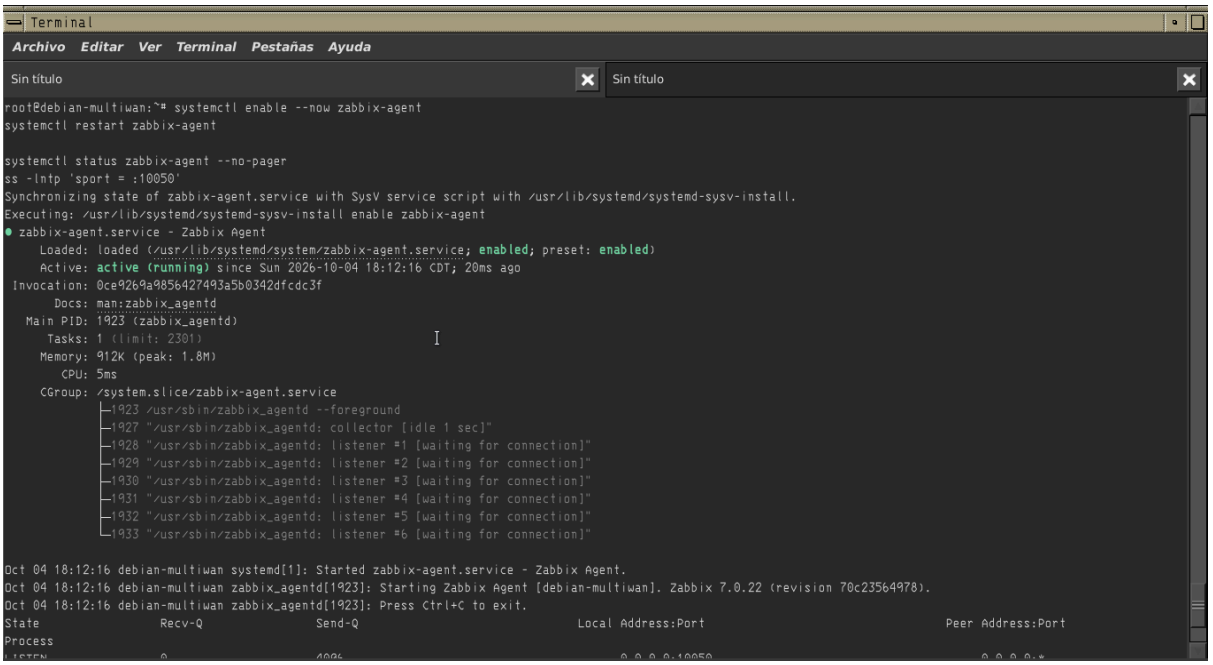
El servicio se habilitó con:

```bash
*sudo systemctl enable --now zabbix-agent*
```

Zabbix obtiene los contadores de entrada y salida de `wan1` y `wan2` mediante las claves `net.if.in[]` y `net.if.out[]`.
instalación de clientes zabbix



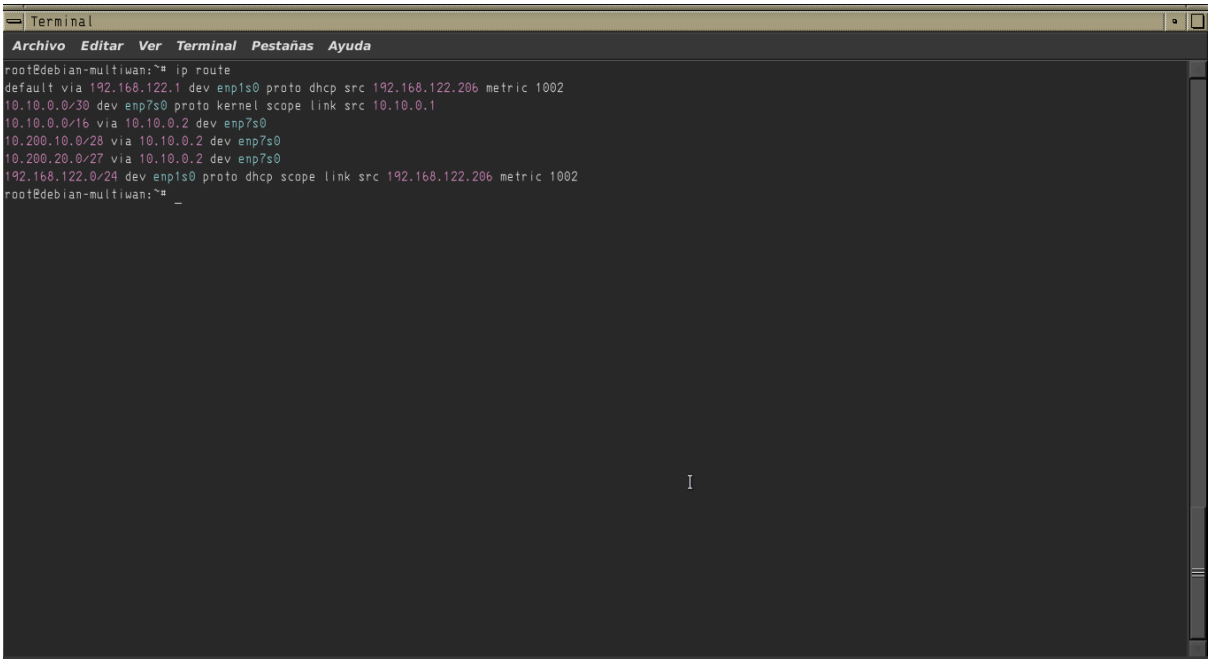
activación del cliente zabbix.



16. Comandos de verificación

16.1 Interfaces y rutas

```bash
*ip -br addr show wan1*
*ip -br addr show wan2*
*ip -br addr show enp7s0*
*ip rule show*
*ip route show table 101*
*ip route show table 102*
```



16.2 Salida individual por cada proveedor

```bash
*ping -4 -n -I wan1 -m 17 -c 3 -W 2 1.1.1.1*
*ping -4 -n -I wan2 -m 18 -c 3 -W 2 1.1.1.1*
```

16.3 Políticas y NAT

```bash
*sudo nft list table ip multiwan_policy*
*sudo nft list table ip multiwan_nat*
*grep -vE '^\s*(#|$)' /etc/multiwan/politicas.conf*
```

16.4 Estado y registros

```bash
*systemctl is-active multiwan nftables zabbix-agent*
*systemctl is-enabled multiwan nftables zabbix-agent*
*sudo journalctl -t multiwan -n 20 --no-pager*
*sudo journalctl -k --since '-5 min' --no-pager | grep MWAN*
```

Archivos de Configuración

ÁRBOL DE ARCHIVOS DE CONFIGURACIÓN DE R-EDGE

```
/
├── etc/
│   ├── network/
│   │   ├── interfaces
│   │   └── interfaces.d/
│   │       ├── multiwan
│   │       └── transito
│   ├── udev/
│   │   └── rules.d/
│   │       └── 70-multiwan-net.rules
│   ├── multiwan/
│   │   ├── mwan.conf
│   │   ├── politicas.conf
│   │   └── playit-egress.nft
│   ├── nftables.conf
│   ├── sysctl.d/
│   │   └── 99-router.conf
│   ├── systemd/
│   │   └── system/
│   │       ├── multiwan.service
│   │       └── playit.service.d/
│   │           └── 50-multiwan.conf
│   ├── playit/
│   │   └── playit.toml
│   └── zabbix/
│       └── zabbix_agentd.conf
├── usr/
│   ├── local/
│   │   └── sbin/
│   │       ├── mwan-apply.sh
│   │       └── mwan-monitor.sh
│   └── lib/
│       └── systemd/
│           └── system/
│               └── playit.service
└── run/
    └── mwan-policy.XXXXXX
```

DESCRIPCIÓN DE LOS ARCHIVOS

/etc/network/interfaces

Es el archivo principal de configuración de red de Debian. Configura la interfaz de loopback, carga los archivos ubicados en /etc/network/interfaces.d/ y define enp1s0 como la interfaz de gestión de la máquina virtual. Esta interfaz se encuentra separada del tráfico principal de XelajuNetwork.

/etc/network/interfaces.d/multiwan

Contiene la configuración IPv4 estática de los dos enlaces hacia Internet. La interfaz wan1 utiliza la dirección 10.38.185.228/24 y el gateway 10.38.185.145, mientras que wan2 utiliza 192.168.42.137/24 y el gateway 192.168.42.129. También crea la ruta por defecto de ISP1 en la tabla 101 y la de ISP2 en la tabla 102, junto con las reglas necesarias para que el tráfico originado en cada dirección WAN responda por su propio proveedor.

/etc/network/interfaces.d/transito

Configura enp7s0 con la dirección 10.10.0.1/30, utilizada para la comunicación entre R-EDGE y el firewall 10.10.0.2. Añade las rutas de retorno hacia las redes internas 10.10.0.0/16 y hacia las redes VPN 10.200.10.0/28 y 10.200.20.0/27. También incorpora reglas para que los destinos internos se consulten en la tabla principal y una ruta inicial por ISP1 para el tráfico que todavía no tenga una marca Multi-WAN.

/etc/udev/rules.d/70-multiwan-net.rules

Contiene las reglas persistentes utilizadas para asignar nombres estables a las interfaces USB de los teléfonos. El teléfono Samsung se identifica como wan1 y el teléfono LG como wan2. Esto evita que los nombres cambien después de desconectar los dispositivos o reiniciar la máquina virtual.

/etc/multiwan/mwan.conf

Define las variables generales utilizadas por los scripts del Multi-WAN. Incluye los nombres de wan1, wan2 y enp7s0, los gateways de ambos proveedores, la dirección 1.1.1.1 empleada para comprobar la conectividad y el intervalo de cinco segundos entre comprobaciones. Centralizar estos valores evita tener que modificar directamente los scripts.

/etc/multiwan/politicas.conf

Almacena las políticas de salida en un archivo externo y modificable. Cada línea indica una dirección o red de origen, uno o varios puertos de destino, el protocolo y el ISP que debe utilizarse. El archivo es procesado por mwan-apply.sh, por lo que es posible agregar, eliminar o modificar políticas sin cambiar el código del sistema.

/etc/multiwan/playit-egress.nft

Contiene las reglas nftables que identifican el tráfico generado por el agente de playit y le asignan una marca para que utilice la infraestructura Multi-WAN. Excluye los destinos internos, las redes VPN, loopback y la red de gestión. También aplica masquerade cuando este tráfico sale por cualquiera de las dos interfaces WAN.

/etc/nftables.conf

Es la configuración principal de nftables en R-EDGE. Define el conjunto de redes internas del proyecto, aplica NAT mediante masquerade al tráfico que sale por wan1 o wan2 y contiene las reglas de reenvío del puerto UDP 51820 hacia VPN-SRV, ubicado en 10.10.50.2. Finalmente incluye las reglas almacenadas en /etc/multiwan/playit-egress.nft.

/etc/sysctl.d/99-router.conf

Contiene los parámetros persistentes del núcleo necesarios para que R-EDGE funcione como router. Su función principal es establecer net.ipv4.ip_forward=1, permitiendo el reenvío de paquetes IPv4 entre el firewall y los proveedores. También desactiva IPv6 porque la infraestructura del proyecto utiliza direccionamiento IPv4.

/etc/systemd/system/multiwan.service

Define el servicio persistente encargado de iniciar el sistema Multi-WAN. Antes de levantar el monitor ejecuta mwan-apply.sh, que carga las políticas y las reglas de balanceo. Después inicia mwan-monitor.sh, que permanece en ejecución comprobando el estado de ambos proveedores. El servicio se reinicia automáticamente si termina debido a un error.

/etc/systemd/system/playit.service.d/50-multiwan.conf

Es un complemento de la unidad de playit. Establece que el agente debe iniciarse después de nftables y del servicio Multi-WAN. Esto garantiza que las marcas, reglas de enrutamiento y NAT estén disponibles antes de que el agente intente establecer su conexión.

/etc/playit/playit.toml

Contiene la configuración privada utilizada por el agente de playit para establecer el punto de entrada público de la VPN. El archivo es leído por el servicio de playit durante su inicio. Debido a que contiene información privada del servicio.

/etc/zabbix/zabbix_agentd.conf

Configura el agente de Zabbix instalado en R-EDGE. Autoriza al servidor 10.10.30.2 para consultar y recibir información del nodo. Mediante este agente se obtienen los contadores de tráfico de entrada y salida de wan1 y wan2, los cuales se muestran en el panel de monitoreo.

/usr/local/sbin/mwan-apply.sh

Es el script encargado de leer y validar /etc/multiwan/politicas.conf. Por cada política genera una regla nftables que asigna la marca 1 para ISP1 o la marca 2 para ISP2. Cuando una conexión no coincide con ninguna política, selecciona aleatoriamente una de las dos marcas para realizar balanceo 50/50 por conexión. También guarda la marca en el seguimiento de conexiones, genera registros y crea las reglas que relacionan las marcas con las tablas de rutas 101 y 102.

/usr/local/sbin/mwan-monitor.sh

Supervisa automáticamente la conectividad de ambos proveedores. Cada cinco segundos realiza una prueba hacia 1.1.1.1, forzada por la interfaz correspondiente. Si un proveedor deja de responder, modifica las reglas para dirigir las dos marcas hacia el ISP disponible. Cuando el enlace se recupera, restaura la asignación normal. Cada cambio queda registrado en el journal con la etiqueta multiwan.

/usr/lib/systemd/system/playit.service

Es la unidad principal instalada por el agente de playit. Ejecuta el proceso con un usuario de servicio independiente, utiliza /etc/playit/playit.toml como archivo de configuración y guarda sus registros en /var/log/playit/. La unidad está configurada para reiniciarse automáticamente si el proceso falla.

/run/mwan-policy.XXXXXX

Es un archivo temporal generado por mwan-apply.sh cada vez que se aplican las políticas. Contiene la tabla nftables construida dinámicamente a partir de politicas.conf. Primero se valida con nft -c; si la sintaxis es correcta, se carga en el sistema. El archivo se elimina automáticamente al finalizar el script.

Pruebas Individuales

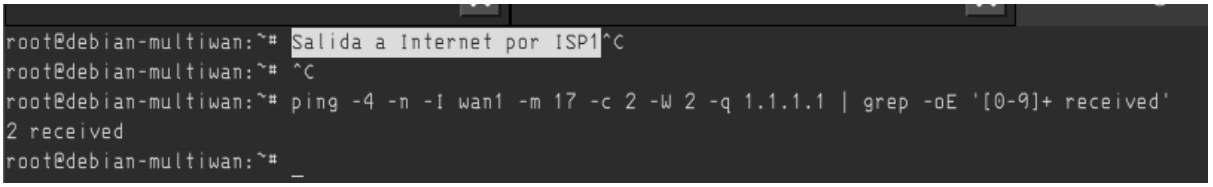
Dos enlaces ISP independientes con direccionamiento manual

ip -br addr show wan1 | awk '{print $1,$3}' | tr '\n' ' '; ip -br addr show wan2 | awk '{print $1,$3}' | tr '\n' ' '; echo "estaticas=$(grep -c 'inet static' /etc/network/interfaces.d/multiwan)"



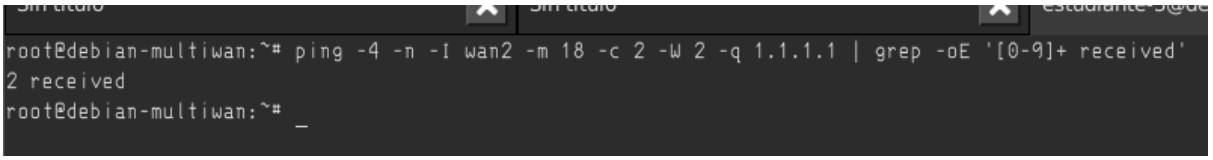
Salida a Internet por ISP1

ping -4 -n -I wan1 -m 17 -c 2 -W 2 -q 1.1.1.1 | grep -oE '[0-9]+ received'



Salida a Internet por ISP2

ping -4 -n -I wan2 -m 18 -c 2 -W 2 -q 1.1.1.1 | grep -oE '[0-9]+ received'



Rutas estáticas: una tabla por ISP



Rutas de retorno hacia la red interna

ip -4 route | grep -E '10\.10\.0\.0/16|10\.200\.' | paste -sd' '



NAT de salida en las dos WAN

nft list table ip multiwan_nat | grep -c masquerade

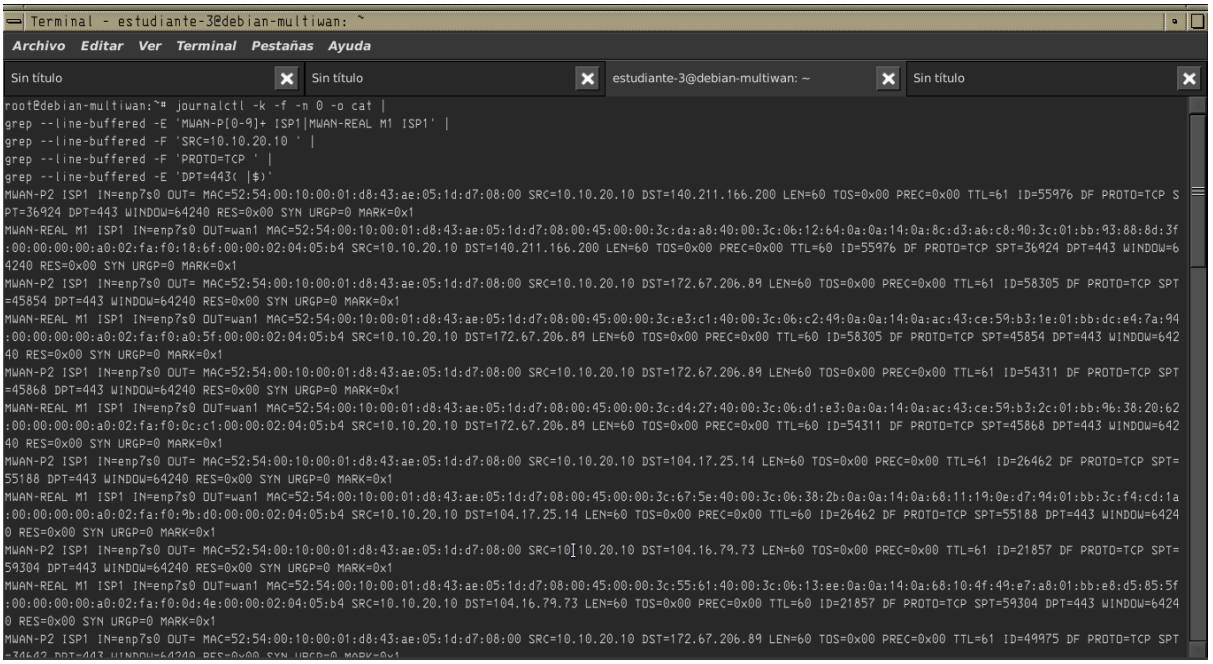
```
root@debian-multiwan:~# nft list table ip multiwan_nat | grep -c masquerade
2
root@debian-multiwan:~# _
```

Políticas en un archivo externo y modificable

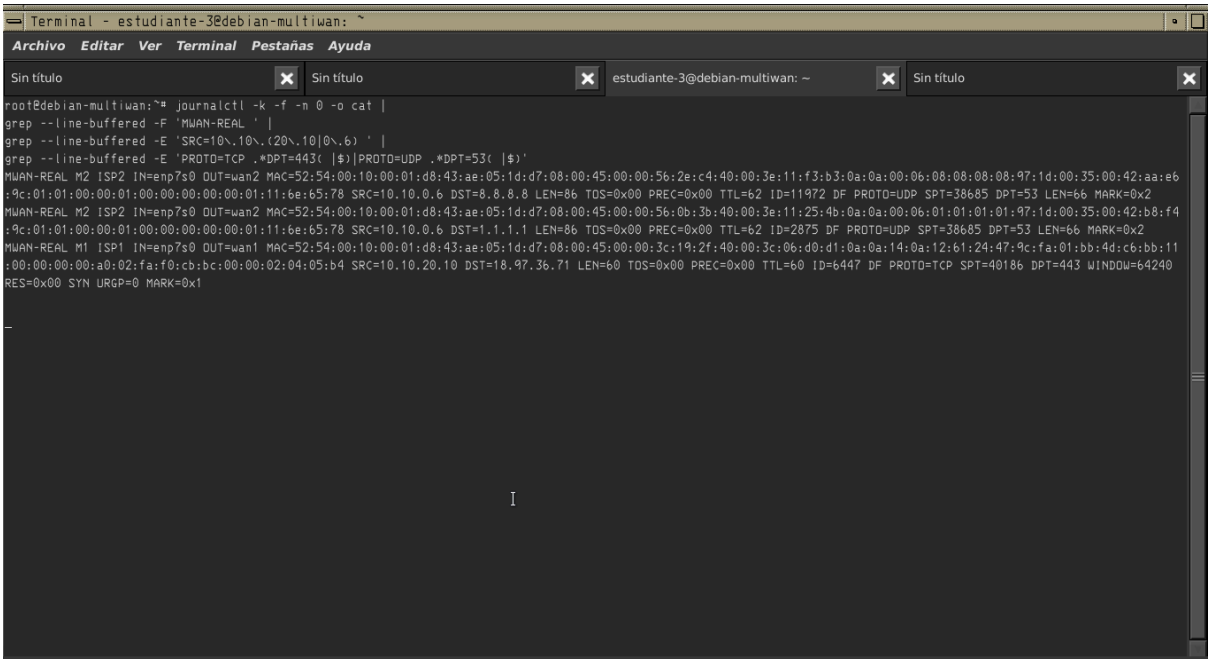
grep -vE '^\s*(#|$)' /etc/multiwan/politicas.conf | wc -l | tr '\n' ' '; nft list chain ip multiwan_policy clasificar | grep -c 'MWAN-P'

```
root@debian-multiwan:~# grep -vE '^\s*(#|$)' /etc/multiwan/politicas.conf | wc -l | tr '\n' ' '; nft list chain ip multiwan_policy clasificar | grep -c 'MWAN-P'
7 7
root@debian-multiwan:~# _
```

Política: HTTPS de USERS por ISP1



Política: DNS por ISP2
journalctl -k -f -n 0 -o cat |
grep --line-buffered -F 'MWAN-REAL ' |
grep --line-buffered -E 'SRC=10\.10\.(20\.10|0\.6) ' |
grep --line-buffered -E 'PROTO=TCP .*DPT=443(|)|$PROTO = UDP.* DPT = 53$(|)'



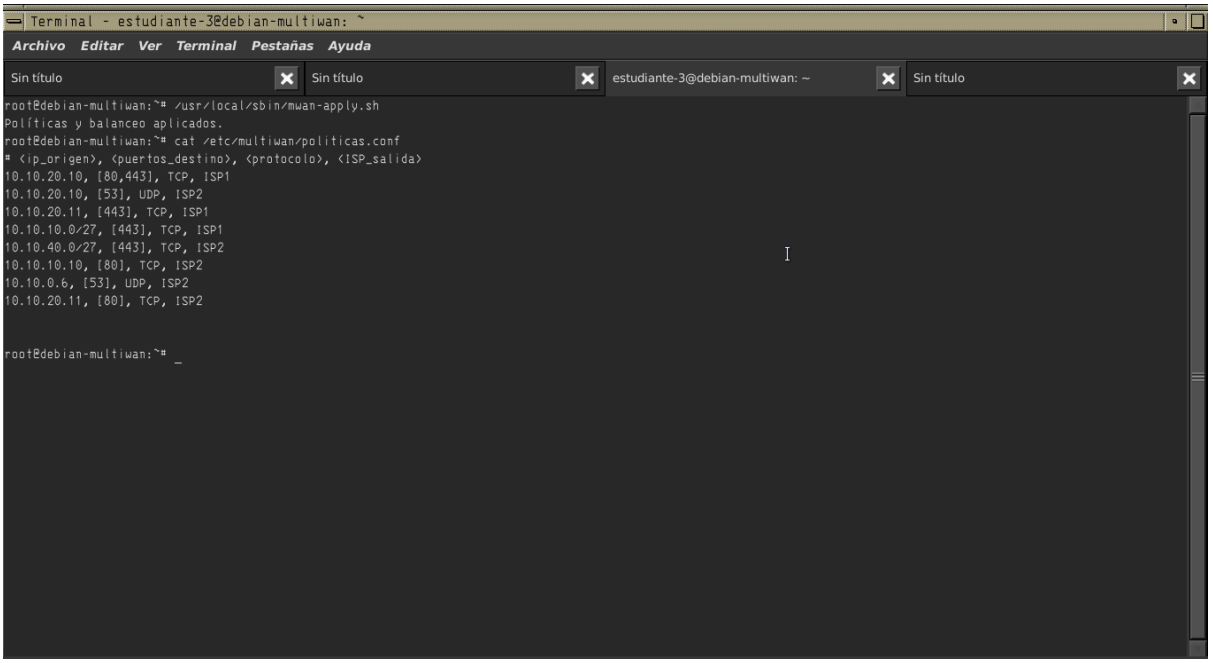
Política: mismo equipo, distinto tráfico por distinto ISP

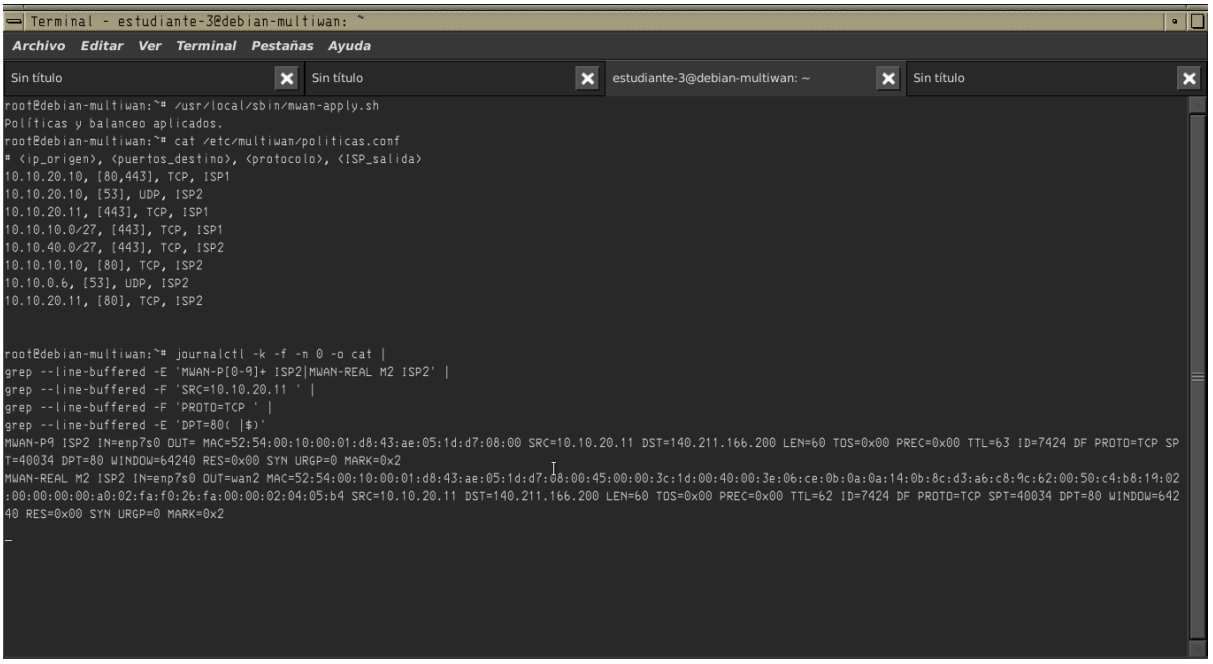
cp -a /etc/multiwan/politicas.conf /etc/multiwan/politicas.conf.bak-prueba

printf '\n10.10.20.11, [80], TCP, ISP2\n' >> /etc/multiwan/politicas.conf

aplicar archivo

/usr/local/sbin/mwan-apply.sh

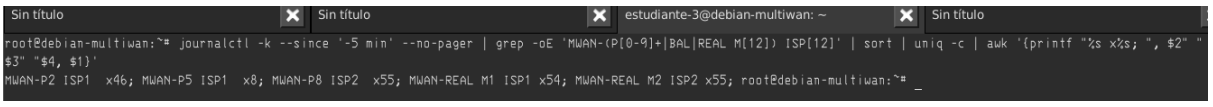




Balanceo cuando no hay política

Registro de la política aplicada y del ISP utilizado

journalctl -k --since '-5 min' --no-pager | grep -oE 'MWAN-(P[0-9]+|BAL|REAL M[12]) ISP[12]' | sort | uniq -c | awk '{printf "%s x%s; ", $2" "$3" "$4, $1}'



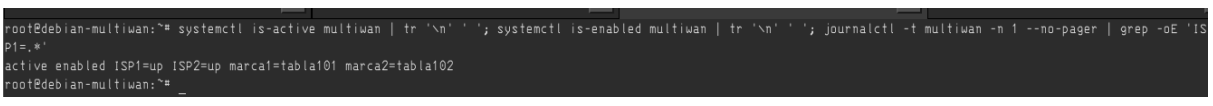
Cambio de política sin modificar el script

Añadir '10.10.20.11, [80], TCP, ISP2' a politicas.conf, ejecutar mwan-apply.sh, probar desde PC-USER02 y revertir

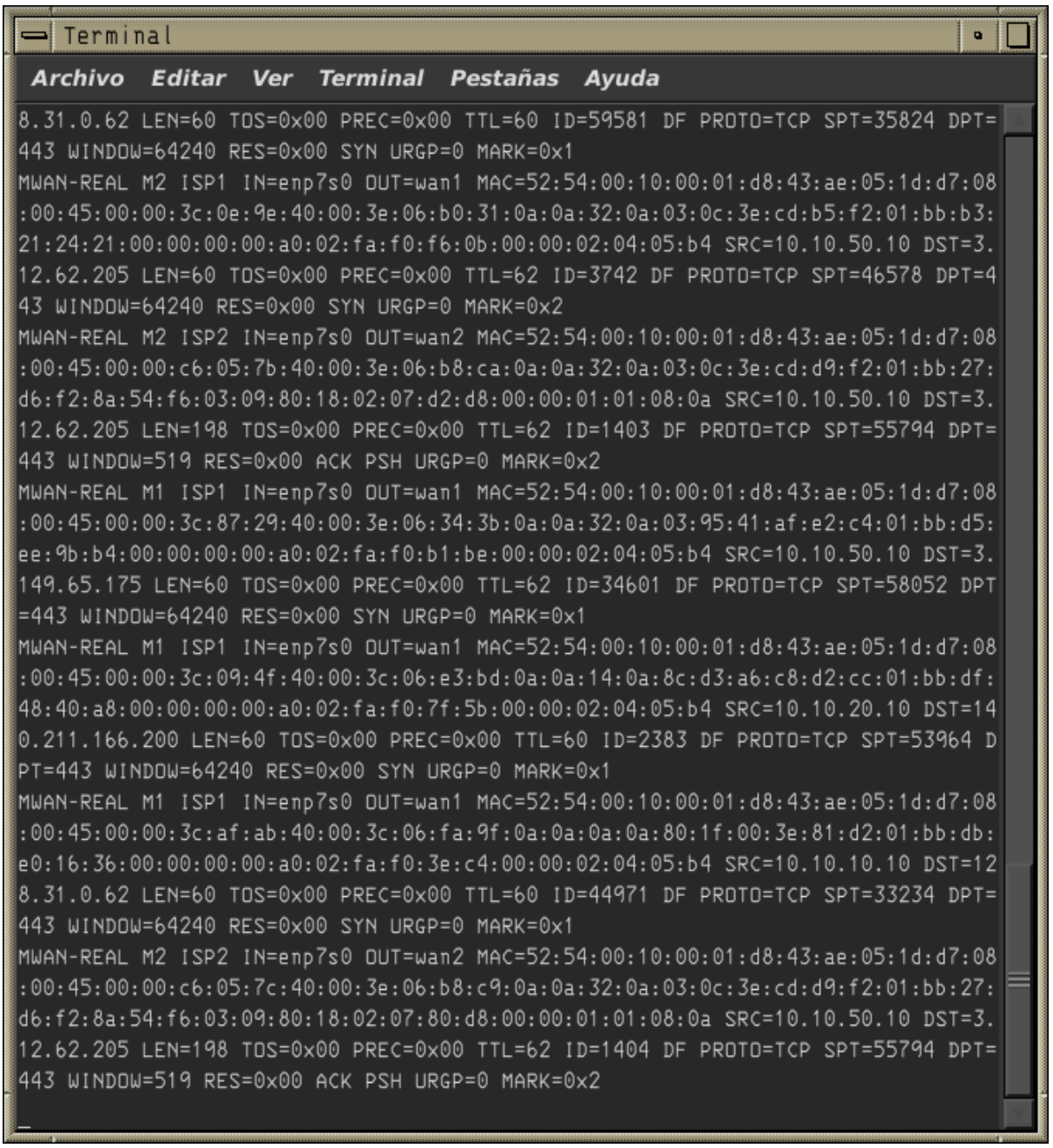
Failover automático y recuperación de los dos ISP

Failover automático y recuperación de los dos ISP

journalctl -t multiwan -f -n 0 -o cat



journalctl -k -f -n 0 -o cat |
grep --line-buffered -F 'MWAN-REAL ' |
grep --line-buffered -F 'PROTO=TCP ' |
grep --line-buffered -E 'DPT=443(|$)'



Métricas de los dos ISP expuestas a Zabbix

```
echo "===== WAN1 - TRAFICO ENTRANTE ====="
zabbix_get -s 10.10.0.1 -k "net.if.in[wan1]"

echo "===== WAN1 - TRAFICO SALIENTE ====="
zabbix_get -s 10.10.0.1 -k "net.if.out[wan1]"

echo "===== WAN2 - TRAFICO ENTRANTE ====="
zabbix_get -s 10.10.0.1 -k "net.if.in[wan2]"

echo "===== WAN2 - TRAFICO SALIENTE ====="
zabbix_get -s 10.10.0.1 -k "net.if.out[wan2]"
'
===== WAN1 - TRAFICO ENTRANTE =====
92062641
===== WAN1 - TRAFICO SALIENTE =====
19589172
===== WAN2 - TRAFICO ENTRANTE =====
6205964
===== WAN2 - TRAFICO SALIENTE =====
4008538
```

Integración de la VPN con el Multi-WAN

```
root@debian-multiwan:~# systemctl is-active playit | tr '\n' ' '; nft list table ip multiwan_nat | grep -c 'dnat to 10.10.50.2:51820'
active 2
root@debian-multiwan:~# _
```